ALC: Adaptive Lossless Compression for LLM Training Checkpoints

Wenqian Li[1], Li Peng[1,2], Weiran Liu[1], Liqiang Peng[1], Zhiqiang Lyu[1], Yan Zhang[1], Wei Wu[1], Lei Zhang[1], Jiamang Wang[1], Lin Qu[1] and Jinfei Liu[2]

[1]Alibaba Group [2]Zhejiang University

liwq62@gmail.com {jerry.pl, weiran.lwr, plq270998}@alibaba-inc.com
{zhiqiang.lv, zy143424, wei.wu}@alibaba-inc.com zongchao.zl@taobao.com
jiamang.wang@alibaba-inc.com xide.ql@taobao.com jinfeiliu@zju.edu.cn

ABSTRACT

Routine checkpointing is essential throughout large language model (LLM) training pipelines, yet storing checkpoints imposes a rapidly growing burden on storage infrastructure due to large checkpoint sizes, massive version counts, and rising storage costs. To reduce storage overhead without disrupting user workflows, a checkpoint archival system seeks to provide strict bit-exactness, high compression efficiency, and low-latency historical serving. However, lossy approaches trade numerical precision for size reduction and cannot guarantee bit-exactness, whereas existing lossless methods overlook the structural and temporal characteristics of training checkpoints and incur long waiting times along sequential delta chains during historical queries.

In this paper, we systematically characterize LLM training checkpoints and identify three key properties governing their compressibility, namely deterministic mixed-precision redundancy, divergent temporal correlation decay across states, and delta inflation under frequent sign flips in first-moment states. Driven by these insights, we propose ALC, an adaptive lossless compression framework. ALC eliminates derivable BF16 weights via verifiable reconstruction and dynamically routes each remaining tensor to its optimal encoding path using fast subsample entropy estimation. At its core, we design RotDelta, a sign-agnostic delta algorithm that rotates the sign bit to the least significant position before differencing to suppress sign-flip inflation. Furthermore, ALC employs a two-level grouping mechanism that bounds delta traversal depth and pipelines parallel decoding with transmission. Evaluations across six LLM architectures and production models up to 235B parameters show that ALC achieves compression ratios down to 50.2% on benchmarks and 56.4% in production, outperforming lossless baselines while accelerating historical serving by up to 3.85×.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/jerryp1/ALC.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

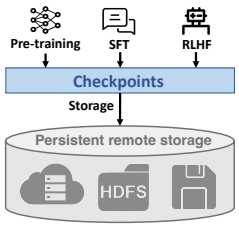


Figure 1: The generation and storage of checkpoints.

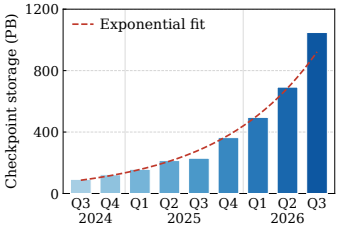


Figure 2: Quarter-end retained checkpoint storage on Alibaba's internal platform (2024 Q3–2026 Q3).

1 INTRODUCTION

Large language models (LLMs) have demonstrated remarkable capabilities through training pipelines that span large-scale pre-training, domain-specific fine-tuning, and reinforcement learning. Throughout the long duration of these training runs, developers periodically save complete snapshots of the training state, known as checkpoints [15, 36, 45, 46, 48, 56, 58], and offload them to persistent remote storage such as Hadoop Distributed File System (HDFS), as shown in Figure 1. These snapshots are essential not only for resuming training after failures [18, 26, 45, 46], but also for evaluating intermediate models [4], launching branched continual training [6], and ensuring historical reproducibility [45].

While essential for lifecycle management and training reliability, this routine checkpointing inevitably translates into substantial storage footprints and financial burdens driven by *large checkpoint sizes*, *massive version counts*, and *rising storage costs*. Under the predominant Adam-family optimizers [25, 32], a complete checkpoint must archive extensive optimizer states (*e.g.*, first- and second-moment states) alongside model parameters. This expands the on-disk footprint to roughly 7× the size of the active BF16 model weights (§3.1). Furthermore, training pipelines retain many historical checkpoints across training steps, while the rapid growth of pre-training and fine-tuning tasks across production clusters [49] further multiplies the total checkpoint count. As a result, aggregate storage demands rapidly escalate to the exabyte scale. As illustrated in Figure 2, the retained checkpoint storage on Alibaba's internal platform has grown from approximately 90 PB in 2024 Q3 to over 1 EB in 2026 Q3. Amplified by surging storage prices [21], this explosive growth places an unsustainable financial burden. Consequently, checkpoint storage compression has emerged as a critical infrastructure requirement.

To reduce checkpoint overhead on storage infrastructure without disrupting user workflows, an archival compression system seeks to achieve three key design goals.

Strict data losslessness. As the foundational data layer, storage infrastructure must preserve user data intact [33, 43, 57], since even minor perturbations to sensitive optimizer states may affect training dynamics and deterministic reproducibility [16, 35, 52]. While lossy checkpoint compressors such as ExCP[29], Inshrinkerator[3], and Amber [50] achieve high in-training compression ratios via quantization or pruning, an independent storage backend cannot alter user data, making bit-exact lossless compression essential.

High compression efficiency. Since massive checkpoint archives are retained for extended periods [45], maximizing storage reduction is critical for lowering archival costs. Existing model-aware lossless compressors (*e.g.*, ZipNN[17], FM-Delta[37], and ZipLLM [49]) and general-purpose compressors (*e.g.*, Zstd[9] and Gzip[11]) target static model weights or generic byte streams without accounting for optimizer states and mixed-precision copies, resulting in suboptimal compression efficiency on checkpoints.

Low-latency checkpoint serving. Storage-side compression should also keep retrieval latency low when users query checkpoints, as slow loading may stall GPU clusters and waste compute resources.

However, achieving these goals simultaneously is non-trivial primarily due to the complex data characteristics of LLM training checkpoints. They encapsulate diverse parameter and optimizer states whose numerical distributions and cross-step evolutions vary widely across state types and training stages (see §3). Lossless compression schemes tuned for a single data pattern tend to be suboptimal on others, making it difficult to extract compact representations across all checkpoint states. Moreover, exploiting cross-checkpoint similarity via delta encoding introduces version dependencies, requiring historical queries to decode multiple predecessor versions and prolonging end-to-end user waiting time.

To bridge this gap, we design ALC, an adaptive lossless compression framework for LLM training checkpoints that simultaneously achieves bit-exactness, high compression efficiency, and low-latency checkpoint serving. Guided by a systematic characterization of checkpoint states, ALC eliminates verifiable mixed-precision redundancy and dynamically routes each remaining tensor to its lowest-cost encoding path, incorporating a novel sign-agnostic delta algorithm named RotDelta to mitigate delta inflation under frequent sign flips. To accelerate historical queries, ALC structures compressed archives via a two-level grouping mechanism that bounds delta traversal depth and pipelines parallel decompression with network transmission. We summarize our contributions below.

- **Checkpoint characterization (§3).** We present an in-depth empirical characterization of LLM training checkpoints across intra- and inter-checkpoint dimensions, revealing the unique data characteristics and compression potential of checkpoint tensors. Our study uncovers three fundamental properties that govern checkpoint compressibility, namely the deterministic reconstructability of BF16 compute weights from FP32 master weights, the pronounced distributional and temporal correlation divergence across parameter types, and the severe compression bottleneck caused by frequent sign flips in first-moment states.
- **Sign-agnostic delta compression algorithm (§4).** To resolve the sign-flip challenge that degrades conventional delta encoding, we design RotDelta, a sign-agnostic delta compression algorithm. By relocating the sign bit to the least significant position and mapping floats into the integer domain before differencing, RotDelta restricts the residual magnitude to the difference rather than the sum. This achieves superior compression ratios on first-moment states and generalizes effectively to other high sign-flip scenarios (§6.6.2).
- **Adaptive compression framework (§5).** To exploit the structural redundancy and temporal heterogeneity of checkpoints, we build ALC, a two-phase lossless compression framework. ALC first eliminates derivable mixed-precision states through a verifiable reconstruction pass, and dynamically routes each remaining tensor to its optimal encoding path (Direct, FM-Delta*, or RotDelta) using fast subsample entropy estimation. Furthermore, ALC incorporates a two-level grouping mechanism that bounds delta traversal depth and pipelines parallel decoding with network transmission for low-latency historical checkpoint serving.
- **Comprehensive evaluation (§6).** We evaluate ALC across six mainstream LLM architectures, production-scale Qwen checkpoints up to 235B parameters at Alibaba Group, and the emerging Muon optimizer. Compared to six SOTA lossless baselines, ALC achieves the best compression ratios of 50.2%–67.7% on standard benchmarks and 56.4%–62.7% on production workloads, while delivering 198.4 MB/s compression and 232.5 MB/s decompression throughput per core. Moreover, we demonstrate that two-level grouping significantly accelerates historical checkpoint serving by up to 3.85×, and that ALC yields substantial economic returns for enterprise archival storage.

2 BACKGROUND AND RELATED WORK

This section provides the essential background on training states and reviews related compression literature.

2.1 LLM Training Checkpoints

In distributed LLM training frameworks such as DeepSpeed [39] and Megatron-LM [42], each rank periodically persists its local checkpoint shard containing model weights, optimizer states, and execution metadata [28, 45]. Model weights and optimizer states are partitioned across ranks and dominate the checkpoint footprint (§3.1), whereas execution metadata tracks lightweight runtime states such as dataloader offsets, random number generator states, and learning rate schedules.

Floating Point Representation of Checkpoints. LLM training commonly adopts mixed-precision strategies, maintaining master weights and optimizer states in FP32 while using reduced formats like BF16 for computation. Under the IEEE 754 standard [22], a floating-point value comprises a sign bit s, an exponent field e, and a mantissa m, encoding $(-1)^s \times 2^{e-\text{bias}} \times (1.m)$ (for normalized values). As illustrated in Figure 3, FP32 allocates 1, 8, and 23 bits to these fields, while BF16 retains the 8-bit exponent with a 7-bit mantissa. We detail LLM checkpoint composition in §3.1.

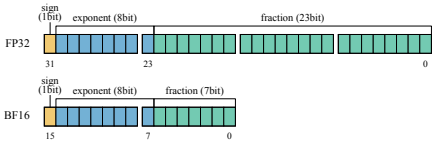


Figure 3: Bit field layout of FP32 and BF16.

2.2 LLM Checkpoint Compression

To mitigate LLM checkpoint storage overhead, existing approaches mainly target lossy compression. ExCP [29] combines non-uniform quantization with joint weight-momentum pruning, Inshrinkerator [3] operates as an in-training compression system that searches for quantization configurations and rearranges parameters for delta encoding, and Amber [50] stores only parameters flagged as changed across steps. Earlier works for traditional DNN and recommendation checkpoints are either lossy [8, 13, 19, 54] or tightly coupled with the active training runtime [13, 14, 54]. In contrast, ALC provides bit-exact archival compression within independent storage without intruding upon the training process.

2.3 LLM Model Compression

Multiple compression methods for LLM model storage reduction have been proposed. ZipNN [17] separates IEEE 754 floating point byte fields and Huffman-codes the exponent stream of model parameters. FM-Delta [37] employs delta encoding to exploit similarity across fine-tuned models. It maps each float to an integer through a piecewise-continuous monotonic map and entropy-codes the integer-domain delta between base and fine-tuned weights. ZipLLM [49] encodes the bitwise XOR residuals between fine-tuned and base weights with Zstd. NeurStore [55] exploits cross-model tensor similarity through lossy delta quantization. QStore [41] efficiently stores multiple precision versions of a model by jointly compressing low-precision weights and the residuals needed for lossless high-precision reconstruction. ALC instead exploits structural and temporal compressibility in checkpoints (§3) with bit-exact recovery.

Table 1: Summary of notations.

Symbol	Description
s, e, m, b	Sign, exponent, and mantissa fields (with bit-widths n_s, n_e, n_m), and word bit-width $b = n_s + n_e + n_m$
$U(x), L(x)$	Unsigned bit pattern of float x, and magnitude field $U(x) \bmod 2^{b-1}$
W, m_t, v_t	FP32 master/BF16 compute weights ($W_{\text{fp32}}, W_{\text{bf16}}$), and Adam moment states
C and $\mathbf{x}$	Checkpoint and tensor
n, η	Saving step interval (Δstep) and sign-flip rate
Δ_m, r_m	Integer-domain delta and effective bit-width under encoding path m
$S, K, B^{(j)}, \widehat{C}_m$	Delta sign, leading-one bit position (BSR), byte column, and estimated bit-cost under encoding path m

2.4 General-purpose Compression

General-purpose coders (Huffman [20], arithmetic/range coding [40, 51], Gzip [11], Zstd [9]) and format-aware floating-point compressors (FPZIP [30], FPC [5], Bitshuffle [34]) treat tensors as opaque streams or exploit only the IEEE 754 layout, yielding limited gains on LLM-related model data, as noted in [37]. Delta compression for general backup storage has been explored in the context of file systems and databases [44, 60], but these methods fail to exploit the numerical characteristics of floating-point tensors, which limits their applicability to checkpoints.

3 CHARACTERIZING LLM TRAINING CHECKPOINTS

We characterize LLM training checkpoint composition (§3.1), the data characteristics of a single checkpoint (§3.2), and the correlation across a sequence of checkpoints (§3.3). These investigations reveal that no single scheme can optimally handle all tensors, motivating the adaptive selection framework ALC and our sign-agnostic Rot-Delta algorithm. Table 1 summarizes the commonly used notations.

3.1 LLM Checkpoint Composition

Modern LLM training pipelines predominantly employ Adam-family optimizers [25, 32]. Under standard mixed-precision training [35], a checkpoint for an M-parameter model comprises $12M$ bytes of FP32 states ($4M$ bytes each for master weights, first-moment states m_t, and second-moment states v_t, stored as `exp_avg` and `exp_avg_sq`) and $2M$ bytes of BF16 compute weights. Figure 4 illustrates this composition across representative models trained under DeepSpeed [39] and Megatron-LM [42], which are the most popular frameworks for training LLMs [27, 47]. Across all architectures and distributed frameworks, the FP32 portion consistently dominates, accounting for $\sim$ 85% of the on-disk footprint.

Mixed-precision structural redundancy. In these training regimes, FP32 master weights preserve accuracy during optimizer updates, while their BF16 copies enable high-throughput forward and backward computation [23, 35]. Because the BF16 tensor is a derived state refreshed from its FP32 master, this conversion typically follows a deterministic mapping such as round-to-nearest, ties-to-even (RNE) $W_{\text{bf16}} = \text{round}^{\text{bf16}}_{\text{RNE}}\left(W_{\text{fp32}}\right)$. While specific conversion rules can vary across implementations, such as truncation or custom rules, these deterministic relationships introduce substantial redundancy within checkpoints.

Observation 1. *Checkpoint composition exhibits two structural properties. (1)* ***Storage dominance****. The FP32 portion accounts for ~85% of the on-disk footprint, establishing master weights and optimizer states as the primary target for compression. (2)* ***Deterministic reconstructability****. Because BF16 compute weights in standard mixed-precision training are refreshed from FP32 masters through deterministic mappings, verified derivable copies can be omitted from storage and regenerated at load time, eliminating $\sim$ 15% of storage without compromising losslessness (§5.2).*

3.2 Intra-Checkpoint States

Within a single checkpoint, different parameter types exhibit heterogeneous numerical distributions. As shown in Figure 6, all three parameter types concentrate within small magnitude ranges near zero. However, they diverge sharply in polarity and dynamic range,

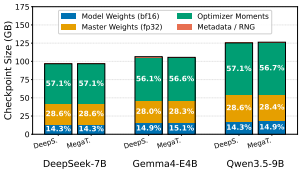


Figure 4: Checkpoint Composition Across Training Frameworks.

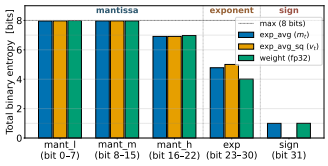


Figure 5: Per-byte Shannon entropy of m_t, v_t and weights in a checkpoint.

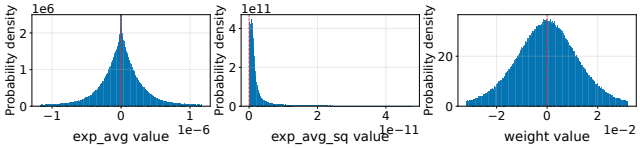


Figure 6: Value distributions of m_t, v_t and model weights.

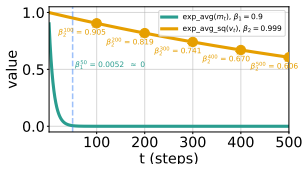


Figure 7: Decay rates of Adam's m_t and v_t.

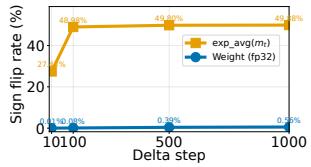


Figure 8: Sign-flip rates of m_t and weights across training steps.

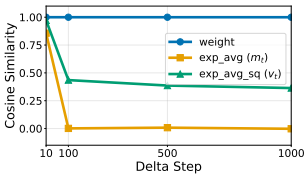


Figure 9: Cosine similarity vs. step interval n.

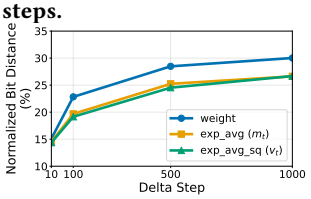


Figure 10: Bit-level distance vs. step interval n.

where model weights and first-moment states m_t are symmetrically distributed around zero, whereas second-moment states v_t are strictly positive. Moreover, m_t spans a much narrower magnitude range than weights ($[-10^{-3}, 10^{-3}]$ vs $[-10^{-1}, 10^{-1}]$), causing m_t to frequently oscillate across zero and incur frequent sign flips (§3.3.2). These parameters follow disparate numerical trajectories, resulting in distinct entropy structures and varying compression potential.

At the representation level, Shannon entropy varies widely across the four constituent bytes of each FP32 element. As illustrated in Figure 5, the compressibility of an FP32 tensor is predominantly determined by its high-order bytes. The sign bit contributes at most 1 bit of entropy for weights and m_t, and exactly 0 bit for v_t due to its non-negativity. The exponent field (sharing the highest two bytes) exhibits substantial redundancy, carrying only 2.1–4.3 bits of entropy per byte. In contrast, the lower mantissa bytes exhibit much higher entropy, hovering between 7.8 and 8.0 bits per byte and offering comparatively limited compression headroom.

Observation 2. *Intra-checkpoint states exhibit two structural characteristics. (1)* ***Distributional heterogeneity****. Weights, m_t, and v_t diverge in polarity and dynamic range, demonstrating that a uniform compression scheme is fundamentally suboptimal and motivating our adaptive selection (§5.3). (2)* ***Byte-field compressibility skew****. Driven by the IEEE 754 bit-field layout, compressibility is predominantly concentrated in high-order bytes containing the sign and exponent, while lower mantissa bits offer narrower compression margins, directly guiding our byte-alignment and leading-zero encoding design (§4.2, §5.3).*

3.3 Inter-Checkpoint Correlation

The compressibility of inter-checkpoint differences depends on their temporal evolution across training steps. Let $\mathbf{x}_t$ and $\mathbf{x}_{t-n}$ denote tensor states separated by step interval n. Because smaller residuals $\Delta_{t-n} = \mathbf{x}_t - \mathbf{x}_{t-n}$ yield lower entropy and tighter delta compression, we analyze inter-checkpoint behavior through temporal correlation decay (§3.3.1) and sign-flip dynamics (§3.3.2).

3.3.1 Temporal Correlation Decay. We study the temporal evolution of checkpoints over training steps. Operating with standard decay rates ($\beta_1 = 0.9, \beta_2 = 0.999$), Adam maintains the first-moment estimate m_t and second-moment estimate v_t as exponential moving averages of the gradients g_t and their squares g_t^2:

$$m_t = \beta_1^n m_{t-n} + (1-\beta_1)\sum_{i=0}^{n-1} \beta_1^i g_{t-i}, \tag{1}$$

$$v_t = \beta_2^n v_{t-n} + (1-\beta_2)\sum_{i=0}^{n-1} \beta_2^i g_{t-i}^2. \tag{2}$$

The coefficient β^n quantifies the correlation between the current optimizer state and its value n steps earlier. As shown in Figure 7, the two moments decay at very different rates, where m_t loses correlation within tens of steps ($\beta_1^{50} \approx 0.005$), while v_t remains correlated over hundreds of steps ($\beta_2^{500} \approx 0.606$).

To further investigate this temporal divergence, we evaluate cosine similarity and bit-level distance across step intervals $n \in \{10, 100, 500, 1000\}$. As shown in Figure 9 and Figure 10, cosine similarity decreases while bit-level distance increases monotonically with n across all parameters. The two moments track their theoretical β^n decay tightly, as m_t decorrelates rapidly (dropping near zero by $n \approx 100$, where $\beta_1^{100} \approx 2.7 \times 10^{-5}$), while v_t decays gradually, falling to 0.364 at $n = 1000$ (cf. $\beta_2^{1000} \approx 0.368$). Model weights behave differently. Their cosine similarity remains nearly invariant (≥ 0.9998), yet their bit-level distance is the highest among all parameters. This occurs because weights concentrate near zero, where tiny numerical updates flip abundant low-order mantissa bits in IEEE 754 representations without rotating the weight vector.

3.3.2 Sign-Flip Characteristics. Beyond magnitude decay, we examine the stability of parameter signs across checkpoint intervals (Figure 8). The sign-flip rate is defined as $\eta = \Pr[\mathrm{sgn}(x_t) \neq \mathrm{sgn}(x_{t-n})]$. Since v_t is always positive ($v_t \geq 0$), its sign never changes regardless of n ($\eta = 0$). Model weights are also stable, with sign-flip rates below 1% throughout training. In contrast, m_t flips much more often, as the decay of β_1^n drives the rate to about 50% for large n, behaving close to a random flip.

Compression Bottleneck under Sign Flips. Frequent sign flips pose a fundamental bottleneck for delta compression. For any element pair (x_t, x_{t-n}) with magnitudes $k_t = |x_t|$ and $k_{t-n} = |x_{t-n}|$, the real-valued difference magnitude $|x_t - x_{t-n}|$ equals the small magnitude difference $|k_t - k_{t-n}|$ when signs match, but jumps to the magnitude sum $k_t + k_{t-n}$ under a sign flip. Because parameter magnitudes change gradually across checkpoints (§3.3.1), this sum is orders of magnitude larger than the difference ($k_t + k_{t-n} \gg |k_t - k_{t-n}|$). Combined with the nearly 50% sign-flip rate of m_t, this zero-crossing inflation dominates delta entropy and severely degrades conventional delta encoders (§4.1).

Observation 3. *Cross-checkpoint evolution reveals two properties. (1)* ***Temporal correlation heterogeneity****. m_t decorrelates within tens of steps, v_t remains correlated over hundreds of steps, and weights stay close in direction throughout training, demonstrating that no uniform compression scheme is optimal and motivating per-tensor adaptive selection (§5). (2)* ***Sign-flip bottleneck****. Because sign flips inflate the delta from $|k_t - k_{t-n}|$ to $k_t + k_{t-n}$, conventional delta encoding degrades, motivating sign-magnitude decoupling in RotDelta (§4).*

4 ROTDELTA: SIGN-AGNOSTIC DELTA COMPRESSION

Building on §3, we present RotDelta, a sign-agnostic delta compression algorithm that resolves the sign-flip bottleneck. We examine how existing delta encoders fail under sign flips (§4.1), detail the RotDelta algorithm which relocates the sign bit prior to differencing (§4.2), and establish a unified bit-width bound that formally caps the sign-flip penalty at a single bit (§4.3).

4.1 Limits of Existing Delta Encoding

Existing model delta compressors [37, 49] target static weight differences rather than rapidly oscillating optimizer states, yielding suboptimal compression under sign flips. Taking FM-Delta [37] as a representative example, its piecewise-continuous mapping (*PCMap*) preserves real-value monotonicity by mapping each b-bit float x with sign $s \in \{0, 1\}$ and magnitude field $L(x) = U(x) \bmod 2^{b-1}$ to

$$\mathrm{PCMap}(x) = \begin{cases} 2^{b-1} + L(x), & s = 0, \\ 2^{b-1} - 1 - L(x), & s = 1. \end{cases} \tag{3}$$

For a corresponding element pair (x_p, x_c) from preceding and current checkpoints with signs (s_p, s_c) and magnitude fields (L_p, L_c), differencing two mapped elements yields an integer residual Δ_{PCMap} that equals $|L_p - L_c|$ when signs match ($s_p = s_c$), but jumps to the sum $L_p + L_c + 1$ under a sign flip ($s_p \neq s_c$). With effective bit-width $\ell(d) = \lfloor \log_2 d \rfloor + 1$ for $d > 0$ and $\ell(0) = 0$, the bit-width $r_{\mathrm{PCMap}} = \ell(\Delta_{\mathrm{PCMap}})$ satisfies

$$r_{\mathrm{PCMap}} \leq \begin{cases} \left\lceil n_m + \log_2(\log_2 R + 2) \right\rceil, & s_p = s_c, \\ b, & s_p \neq s_c, \end{cases} \tag{4}$$

where $R = \frac{\max(|x_p|,|x_c|)}{\min(|x_p|,|x_c|)}$ is the magnitude ratio (derivation in Appendix A [1]). While the same-sign branch stays tightly bounded by R, the sign-flip branch saturates at the full word width b, destroying leading-zero sparsity and degrading delta compression.

4.2 RotDelta Algorithm Design

Overview. To prevent sign flips from saturating bit-widths and degrading compressibility, we propose RotDelta, a delta compression algorithm that realizes sign-magnitude decoupling across two sequential stages. The first stage is a preprocessing mapping (*RotMap*) that relocates the sign bit from the most significant bit (MSB) to the least significant bit (LSB), aligning positive and negative values onto a shared magnitude axis. The second stage applies a leading-zero encoder that exploits the resulting near-zero sparsity via tabular Asymmetric Numeral Systems (tANS) [12]. Algorithm 1 summarizes the complete procedure, and we detail each stage below.

Algorithm 1: RotDelta: Sign-Agnostic Delta Compression

Input: two checkpoints C_p, C_c; tANS encoder $\mathcal{M}$
Output: compressed $\hat{C}_p$; uncompressed C_c

1: **for each** tensor pair $(\mathbf{x}_p, \mathbf{x}_c)$ from (C_p, C_c) **do**
2: **for each** element pair (x_p, x_c) from $(\mathbf{x}_p, \mathbf{x}_c)$ **do**
3: $u_p \leftarrow \mathrm{RotMap}(x_p)$; $u_c \leftarrow \mathrm{RotMap}(x_c)$
4: $S \leftarrow \mathrm{sgn}(u_c - u_p)$; $\Delta_{\mathrm{RotMap}} \leftarrow |u_c - u_p|$
5: **if** $\Delta_{\mathrm{RotMap}} > 0$ **then** $K \leftarrow \lfloor \log_2 \Delta_{\mathrm{RotMap}} \rfloor$ **else** $K \leftarrow 0$
6: cp $\leftarrow$ cp $\|$ $\mathcal{M}$.encode(S, K) ▷ *tANS-encode the joint symbol*
7: rr $\leftarrow$ rr $\|$ $(\Delta_{\mathrm{RotMap}} \bmod 2^K)$ ▷ *lowest K raw bits*
8: **end for**
9: $\hat{\mathbf{x}}_p \leftarrow$ cp $\|$ rr ▷ cp: ***c**ompressed **p**art;* rr: ***r**aw **r**emainder*
10: **end for**
11: $\hat{C}_p \leftarrow \{\hat{\mathbf{x}}_p\}_{\mathbf{x}_p \in C_p}$
12: **return** compressed $\hat{C}_p$; uncompressed C_c

Preprocessing map. The preprocessing stage of RotDelta (Algorithm 1, Lines 3–4) realizes sign-magnitude decoupling via *RotMap*. For each floating-point element x, we reinterpret its IEEE 754 bit pattern as an unsigned integer $U(x)$ and apply a 1-bit circular left shift to rotate the sign bit into the vacated LSB

$$\begin{aligned} u = \mathrm{RotMap}(x) &= 2L(x) + s(x) \\ &= \big((U(x) \ll 1) \mid (U(x) \gg (b-1))\big) \bmod 2^b, \end{aligned} \tag{5}$$

where b=32 for FP32, and $L(x) = U(x) \bmod 2^{b-1}$ is the magnitude-field integer. Because RotMap is a circular 1-bit shift, it directly compiles into a single CPU rotate instruction (e.g., `ROL` on x86 or `ROR` on ARM) with zero branching overhead. Applying this mapping to an element pair yields $u_p = \mathrm{RotMap}(x_p)$ and $u_c = \mathrm{RotMap}(x_c)$, from which we compute the absolute integer delta

$$\Delta_{\mathrm{RotMap}} = |u_c - u_p|. \tag{6}$$

Differencing two mapped values yields $u_c - u_p = 2(L_c - L_p) + (s_c - s_p)$. Because the sign perturbation is strictly confined to $(s_c - s_p) \in \{-1, 0, 1\}$, sign flips no longer trigger MSB-dominated delta inflation. Instead, the residual is governed by the magnitude difference $|L_c - L_p|$ rather than the magnitude sum $L_p + L_c + 1$, preserving leading-zero sparsity for downstream compression.

Applying leading-zero encoder. The second stage of RotDelta (Algorithm 1, Lines 5–7) emits a compressed bit-stream from the non-negative Δ_{RotMap} values. Because RotMap eliminates sign-flip magnitude spikes, the resulting deltas concentrate tightly near zero, exposing many leading zeros in their binary representations. We

thus encode the position of the leading non-zero bit and concatenate the remaining low-order bits as a raw bit-stream.

For each delta Δ_{RotMap}, we extract two parts, consisting of a joint symbol (S, K) and a raw remainder. Here $S = \text{sgn}(u_c - u_p) \in \{-1, 0, +1\}$ is the sign of the integer delta and $K = \lfloor \log_2 \Delta_{\text{RotMap}} \rfloor \in [0, b-1]$ (set to 0 if $\Delta_{\text{RotMap}} = 0$) is the zero-indexed leading non-zero bit position. When $\Delta_{\text{RotMap}} > 0$, the leading non-zero bit at position K is implicitly one and requires no transmission, while the lowest K bits of Δ_{RotMap} are stored as raw bits. Notably, the joint symbol (S, K) spans a small alphabet of $2b{+}1$ states where b is the bit-width (e.g., 2×32+1 = 65 for FP32) and captures the statistical coupling between update direction and magnitude. We then encode (S, K) near its joint entropy $H(S, K)$ using tANS [12] compressor.

Why tANS? We choose tANS over other arithmetic/entropy coding (*e.g.*, range coding used in [37]) because it achieves a compression ratio close to the Shannon entropy limit while delivering higher throughput via table lookups, especially on small alphabets like our setting (typically lower than 65). We use FM-Delta* to denote PCMap-based delta compression [37] with tANS as the coding backend in the following sections.

Geometric intuition. Figure 11 illustrates the sign-flip pair $x_p \approx -0.12$ and $x_c \approx +1.27$, with FP32 bit patterns $U(x_p) =$ `0xBDF5C291` and $U(x_c) =$ `0x3FA28F5E`. PCMap places them on opposite sides of 2^{31}, at distances d_- and d_+, yielding $\Delta_{\text{PCMap}} = d_- + d_+$. RotMap places them at $2d_- - 1$ and $2d_+$ from zero, yielding $\Delta_{\text{RotMap}} = |2d_+ - (2d_- - 1)|$. This replaces the distance sum with a difference, reducing the residual from `0x7D9851F0` (31 bits) to `0x03599999` (26 bits) and exposing five additional leading zeros to allow more efficient encoding.

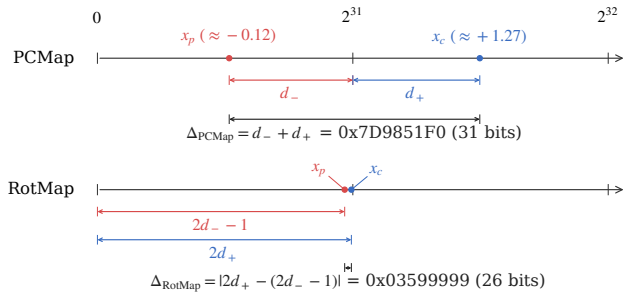


Figure 11: PCMap and RotMap for the sign-flip pair $x_p \approx -0.12$ and $x_c \approx +1.27$.

4.3 Theoretical Analysis

We analyze the advantage of RotDelta by establishing a unified, sign-agnostic upper bound on effective bit-width $r_{\text{RotMap}} = \ell(\Delta_{\text{RotMap}})$.

Unified bit-width bound. Directly from (5), any element pair satisfies $|u_c - u_p| \le 2|L_p - L_c| + 1$, regardless of whether a sign flip occurs. Adding this overhead to the same-sign branch of Equation 4 yields a unified, sign-agnostic upper bound

$$r_{\text{RotMap}} \le \min\left\{b, \left\lceil n_m + 1 + \log_2(\log_2 R + 2)\right\rceil\right\} \tag{7}$$

which holds for either sign combination (derivation in Appendix B.1 [1]). Equivalently, Δ-RotMap eliminates Δ-PCMap's $(n_s + n_e + n_m)$-bit sign-flip case and replaces the two-branch bound with a single unified bound, capping the sign-flip penalty at a single bit.

Proposition 1. *For any finite opposite-sign pair (x_p, x_c) satisfying $\max(L_p, L_c) < 3\min(L_p, L_c)$, the absolute integer residual under RotMap is strictly smaller than under PCMap, i.e., $\Delta_{RotMap} < \Delta_{PCMap}$.*

The proof is in Appendix B.2 [1]. This smaller residual exposes more leading zeros and compression potential under sign flips, which ALC selectively exploits via cost-driven path selection (§5).

5 ALC: ADAPTIVE LOSSLESS COMPRESSION

5.1 Overview

Driven by the structural redundancy and temporal divergence characterized in §3, we propose ALC, an adaptive lossless compression framework that builds on our sign-agnostic RotDelta algorithm (§4) and orchestrates both redundancy removal and cost-driven multi-path compression across heterogeneous tensors.

Compression workflow. ALC executes checkpoint compression in two sequential phases, as detailed in Algorithm 2 and illustrated in Figure 12. Guided by the deterministic reconstructability of mixed-precision representations (Observation 1), Phase 1 (Structural Redundancy Removal) verifies deterministic derivation rules against master weights, omitting reproducible tensors such as BF16 compute copies from serialization (§5.2). For the remaining FP32 states, Phase 2 (Adaptive Compression) accommodates the distributional and temporal divergence across parameters (Observations 2 and 3) through per-tensor path selection. Specifically, Phase 2.a (Adaptive Path Selection) evaluates candidate representations on deterministic subsamples to pick the lowest-cost path $m^* \in \{\text{Direct}, \text{FM-Delta}^*, \text{RotDelta}\}$ (§5.3.1), after which Phase 2.b (Adaptive Encoding) applies the selected transformation and streams the payload into a shared tANS backend (§5.3.2). By combining structural redundancy removal with adaptive path selection and encoding, ALC maximizes checkpoint storage savings across diverse training workloads while guaranteeing lossless recovery and exact reproducibility.

Archival and serving. To organize the resulting archive across training steps, ALC retains the latest checkpoint $C^{(T)}$ uncompressed as the root anchor (line 2 in Algorithm 2), ensuring instant resumption for training recovery. Historical checkpoints are structured and served via a *two-level grouping* mechanism to achieve fast random query response. At the version level, ALC forms ckpt groups with Direct-encoded anchors to bound the reverse delta recovery depth within $g_{\text{ckpt}} - 1$ hops. At the data stream level, ALC forms chunk groups of size g_{chunk} to pipeline decompression across worker threads and network transmission. By overlapping compute with network I/O, this two-level design substantially reduces end-to-end retrieval latency for historical checkpoints (§5.4).

5.2 Structural Redundancy Removal

Phase 1 removes structural redundancy from checkpoints before entropy coding (line 5 in Algorithm 2). While BF16 compute weights are typically derived from FP32 master weights via deterministic rules such as RNE or truncation (§3.1), corner cases like stochastic rounding or custom kernels can diverge. To guarantee bit-exactness

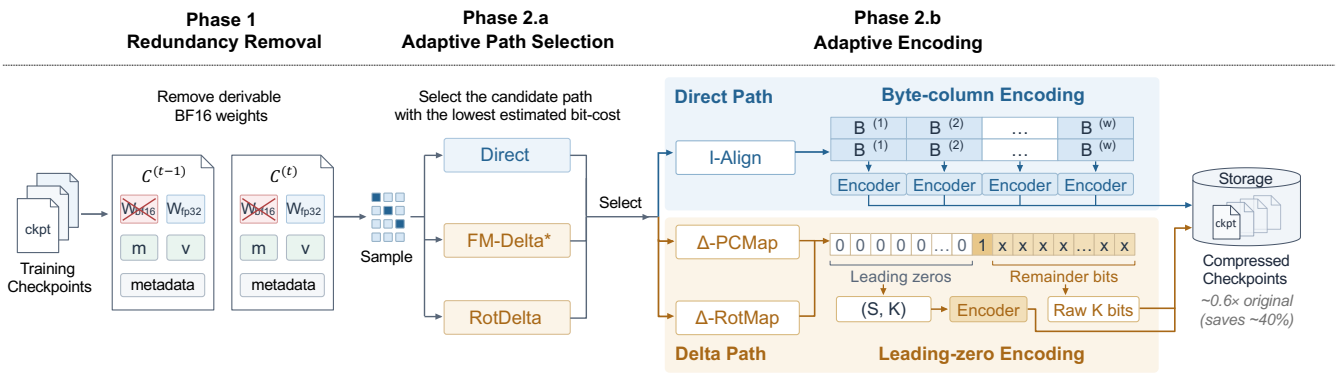


Figure 12: ALC overview.

Algorithm 2: Adaptive Compression Selection Pipeline

Input checkpoints $C^{(0)}, \ldots, C^{(T)}$, tANS encoder $\mathcal{M}$
Output $C^{(T)}$ uncompressed, compressed $\hat{C}^{(0)}, \ldots, \hat{C}^{(T-1)}$
1: $\mathcal{P} \leftarrow \{\text{Direct}, \text{FM-Delta}^*, \text{RotDelta}\}$ ▷ *Candidate path set*
2: Store $C^{(T)}$ uncompressed as the head of the reverse delta chain
3: **for** $t = 1$ **to** T **do**
4: ▷ *Phase 1 (Structural Redundancy Removal)*
5: Verify derivable tensors and omit verified copies
6: ▷ *Phase 2 (Adaptive Compression)*
7: **for each** remaining tensor $\mathbf{x}_p \in C^{(t-1)}$ **do**
8: Let $\mathbf{x}_c \in C^{(t)}$ be the reference tensor corresponding to $\mathbf{x}_p$
9: ▷ *Phase 2.a (Adaptive Path Selection)*
10: $J \leftarrow$ deterministic indices with $|J| = \min(|\mathbf{x}_p|, 65{,}536)$ ▷ *Subsampling*
11: $\mathbf{d}_{\text{Direct}} \leftarrow \text{I-Align}(\mathbf{x}_p[J])$, estimate $\widehat{C}_{\text{Direct}}$ via (8) ▷ *Direct cost*
12: **if** $\mathbf{x}_c$ exists **then**
13: $\mathbf{d}_{\text{FM-Delta}^*} \leftarrow \text{PCMap}(\mathbf{x}_c[J]) - \text{PCMap}(\mathbf{x}_p[J])$
14: $\mathbf{d}_{\text{RotDelta}} \leftarrow \text{RotMap}(\mathbf{x}_c[J]) - \text{RotMap}(\mathbf{x}_p[J])$
15: Estimate $\widehat{C}_{\text{FM-Delta}^*}, \widehat{C}_{\text{RotDelta}}$ via (9) ▷ *Delta costs*
16: **else**
17: Set $\widehat{C}_{\text{FM-Delta}^*} \leftarrow \infty, \widehat{C}_{\text{RotDelta}} \leftarrow \infty$
18: $m^* \leftarrow \arg\min_{m \in \mathcal{P}} \widehat{C}_m$ ▷ *Optimal choice*
19: ▷ *Phase 2.b (Adaptive Encoding)*
20: **if** $m^* = \text{Direct}$ **then**
21: Apply I-Align$(\mathbf{x}_p)$ and split stream into w byte columns
22: Encode each byte column separately via encoder $\mathcal{M}$ ▷ *Direct Compression*
23: **else**
24: $\mathbf{d} \leftarrow \text{PCMap}(\mathbf{x}_c) - \text{PCMap}(\mathbf{x}_p)$ if $m^* = \text{FM-Delta}^*$
else $\text{RotMap}(\mathbf{x}_c) - \text{RotMap}(\mathbf{x}_p)$ ▷ *Delta Compression*
25: **for each** residual $d \in \mathbf{d}$ **do**
26: $S \leftarrow \text{sgn}(d)$ and $K \leftarrow \lfloor \log_2 |d| \rfloor$ if $d \neq 0$, else 0
27: Encode joint symbol (S, K) with $\mathcal{M}$ and write lowest K raw bits
28: Write encoded payload and metadata for tensor $\mathbf{x}_p$
29: **return** $C^{(T)}$ uncompressed, compressed $\hat{C}^{(0)}, \ldots, \hat{C}^{(T-1)}$

without unverified runtime assumptions, ALC tests each shape-compatible BF16 and FP32 pair against candidate deterministic rules. If a rule reproduces the BF16 bits identically across all elements, ALC records the rule identifier in the archive manifest and omits the BF16 tensor for on-the-fly regeneration during decompression. Otherwise, ALC safely falls back to compressing the BF16 tensor losslessly. This verification pass accounts for only 2.4% of end-to-end runtime (§6.3). In LoRA-style or partial fine-tuning, frozen tensors identical across checkpoints are stored only once in the segment's uncompressed checkpoint, with earlier copies replaced by compact references.

5.3 Adaptive Compression

For all tensors remaining after Phase 1, Phase 2 performs tensor-level adaptive compression to exploit the parameter heterogeneity identified in §3.2 and §3.3. This stage couples fast candidate path selection on small deterministic subsamples (§5.3.1) with full tensor transformation and encoding under the selected path (§5.3.2).

5.3.1 Adaptive Path Selection. To maximize storage savings, ALC aims to route each tensor to the encoding scheme that yields the most compact representation. However, running exhaustive trial compression for every tensor across candidate paths would incur substantial computational overhead. To address this, Phase 2.a evaluates candidate representations on deterministic subsamples and estimates encoded bit costs prior to full compression (lines 10-18 in Algorithm 2).

Deterministic subsampling. Deterministic subsampling extracts distribution statistics from compact element buffers without transforming the entire tensor. For a target tensor $\mathbf{x}_p$ and its reference tensor $\mathbf{x}_c$, ALC evaluates candidate representations on a shared deterministic index set J, preserving element correspondence across checkpoints. The sample size is bounded by $|J| = \min(|\mathbf{x}_p|, 65{,}536)$ elements, where $|\mathbf{x}_p|$ is the element count of $\mathbf{x}_p$ (line 10 in Algorithm 2). When no shape-compatible reference tensor exists in the reference checkpoint, delta paths are disqualified and Direct compression becomes the sole eligible candidate.

Bit-cost model. To make direct and delta paths comparable, our cost model estimates the expected encoding cost of each candidate in bits per element (lines 11 and 15 in Algorithm 2).

For the direct path, the cost model takes the target tensor subsample $\mathbf{x}_p[J]$ as input. The I-Align transformation rearranges floating-point word fields from $[s \mid e \mid m]$ to $[e \mid m \mid s]$, moving the sign bit to the least significant position to concentrate the low-entropy exponent bits into a single byte and improve byte-level compression ratios (§3.2). This aligns the 8-bit exponent of FP32 and BF16 with byte boundaries, splitting the sampled stream into w byte columns $B^{(1)}, \ldots, B^{(w)}$, where $w = 4$ for FP32 and $w = 2$ for BF16. Because each byte column is compressed independently in tANS, the direct bit-cost estimate sums their empirical Shannon entropies, expressed

as

$$\widehat{C}_{\text{Direct}}(\mathbf{x}_p[J]) = \sum_{j=1}^{w} \hat{H}(B^{(j)}), \tag{8}$$

where $\hat{H}(B^{(j)})$ denotes the empirical Shannon entropy computed over the j-th byte column $B^{(j)}$ derived from $\mathbf{x}_p[J]$.

For each delta candidate $m \in \{\text{FM-Delta}^*, \text{RotDelta}\}$, the cost model takes the target and reference subsample pair $(\mathbf{x}_p[J], \mathbf{x}_c[J])$ as input. It computes the sampled integer residual vector $\mathbf{d}_m = \Phi_m(\mathbf{x}_c[J]) - \Phi_m(\mathbf{x}_p[J])$, where $\Phi_{\text{FM-Delta}^*}$ = PCMap preserves numerical order (§4.1) and Φ_{RotDelta} = RotMap suppresses magnitude inflation under sign flips (§4.2). For each residual element $d \in \mathbf{d}_m$, the encoder extracts the sign $S = \text{sgn}(d)$ and the leading-one bit position $K = \lfloor \log_2 |d| \rfloor$ when $d \neq 0$, setting $(S, K) = (0, 0)$ when $d = 0$. The estimated delta bit cost combines the joint entropy of (S, K) with the expected length of uncoded remainder bits, given by

$$\widehat{C}_m(\mathbf{x}_p[J], \mathbf{x}_c[J]) = \hat{H}(S, K) + \widehat{\mathbb{E}}[K], \tag{9}$$

where $\hat{H}(S, K)$ represents the empirical joint Shannon entropy of the symbol stream and $\widehat{\mathbb{E}}[K]$ denotes the sample average of raw remainder bits per element in $\mathbf{d}_m$.

Selection rule. With candidate set $\mathcal{P} = \{\text{Direct}, \text{FM-Delta}^*, \text{RotDelta}\}$, ALC chooses the path that minimizes the estimated bit cost, defined by

$$m^* = \arg\min_{m \in \mathcal{P}} \widehat{C}_m. \tag{10}$$

When no reference tensor exists, the delta path costs are initialized to infinity, making Direct the sole candidate (lines 17 in Algorithm 2). This cost-driven selection identifies the optimal encoding path with sub-millisecond evaluation latency, avoiding trial compression overhead.

5.3.2 Adaptive Encoding. Once the optimal path m^* is chosen, Phase 2.b applies the winning transformation to tensor $\mathbf{x}_p$ and streams the symbols into the tANS entropy backend (lines 20-28 in Algorithm 2).

Direct encoding execution. When m^* = Direct, ALC compresses the tensor independently without referencing historical checkpoints. It applies the I-Align transformation to tensor $\mathbf{x}_p$ and splits the output into w byte columns (lines 21, 22 in Algorithm 2). Each byte column is compressed independently with tANS encoding. During restoration, the decoded columns are interleaved back into words and the circular bit shift is inverted to reconstruct the original IEEE 754 values bit-exactly.

Delta encoding execution. When m^* is FM-Delta* or RotDelta, the encoder computes the integer difference $\mathbf{d}$ between target tensor $\mathbf{x}_p$ and reference $\mathbf{x}_c$ (lines 24-27 in Algorithm 2). Each residual d is decomposed into joint symbol (S, K) and raw remainder bits $q = |d| \bmod 2^K$. The sequence of (S, K) symbols is encoded by tANS near its joint entropy limit, while the lowest K remainder bits are emitted directly into a raw bit-stream without entropy coding. During restoration, the decompressor reconstructs $d = S(2^K + q)$ when $K > 0$, restores the reference value, and inverts the integer mapping to achieve lossless recovery.

Archive packaging and metadata. ALC writes the compressed byte stream together with compact tensor metadata, recording the path identifier m^*, tensor dimensions, and reference dependencies (line 28 in Algorithm 2). This lightweight metadata enables the restoration engine to directly dispatch the corresponding inverse transformation.

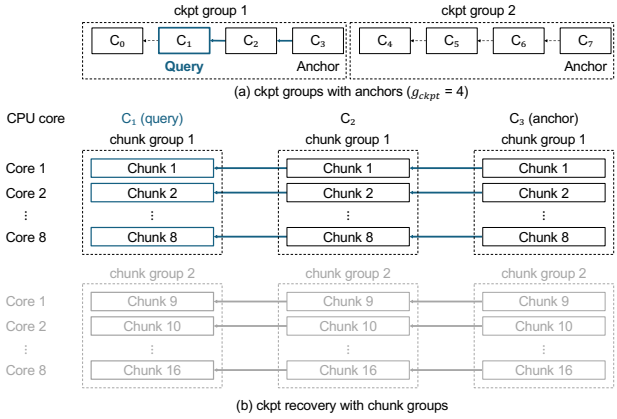


Figure 13: Two levels of grouping for checkpoint serving. (a) ckpt groups bound recovery depth to at most $g_{\text{ckpt}} - 1$ hops using anchors. (b) chunk groups enable parallel recovery of corresponding chunks across versions.

5.4 Checkpoint Serving

Queries for historical checkpoints deep in a reverse delta chain can incur long waits before receiving data, because the recovery path may span the entire chain and serial restoration delays transmission until full recovery finishes. We address these two challenges with *two levels of grouping*.

Bounding recovery depth with ckpt groups. To bound recovery depth, we partition consecutive checkpoints into *ckpt groups* of at most g_{ckpt} versions. While the latest checkpoint $C^{(T)}$ remains uncompressed for instant failure recovery, each historical group's newest version serves as a local anchor compressed via Direct encoding. Remaining versions compress backward with delta references confined within their group. A historical query therefore starts at its group anchor and traverses at most $g_{\text{ckpt}} - 1$ hops, as illustrated in Figure 13(a).

Pipelining recovery with chunk groups. To enable early output, we process corresponding tensor ranges across versions in chunks of c elements. Consecutive chunks form a *chunk group* of size g_{chunk}, with one chunk assigned to each worker thread. Threads consume the decoded residuals to reconstruct corresponding chunks along the version dependencies until the target checkpoint is reached. Restored chunks are transmitted in order as they become available while threads advance to the next chunk group, as depicted in Figure 13(b). This pipelining overlaps compute with network I/O, substantially reducing time-to-first-byte latency and delivering query response speeds nearly identical to uncompressed storage systems when parallel decoding throughput outpaces network bandwidth (see §6.4).

Proposition 2. *For a checkpoint of size V with compression ratio r, element byte width w, group size g_{ckpt}, chunk size c, $P = g_{chunk}$ decoding threads, storage read bandwidth B_{io}, per-core decoding throughput B_d, and network bandwidth B_{net}, whenever*

$$\frac{1}{g_{ckpt}} \min\left(\frac{B_{io}}{r},\ PB_d\right) \ \geq\ B_{net}, \tag{11}$$

network transmission completely hides storage reading and decoding after the first chunk, bounding the end-to-end serving time by $T_{serve} \leq \frac{V}{B_{net}} + g_{ckpt}cw\left(\frac{rP}{B_{io}} + \frac{1}{B_d}\right)$.

In this pipeline, reading and decoding a chunk group across at most g_{ckpt} versions take $t_{\text{io}} \leq \frac{g_{\text{ckpt}}rPcw}{B_{\text{io}}}$ and $t_{\text{dec}} \leq \frac{g_{\text{ckpt}}cw}{B_d}$, while network transmission takes $t_{\text{send}} = \frac{Pcw}{B_{\text{net}}}$. Condition (11) ensures $\max(t_{\text{io}}, t_{\text{dec}}) \leq t_{\text{send}}$, hiding both storage I/O and decoding behind transmission after the first chunk.

6 EVALUATION

In this section, we conduct a comprehensive experimental evaluation of ALC. We aim to answer the following research questions:

- **RQ1: Overall Performance.** How does ALC improve end-to-end compression ratio and throughput compared to baselines across diverse LLM configurations? (§6.2)
- **RQ2: Ablation and Design Analysis.** How do ablation configurations isolate the gains of individual components, and how effectively do core designs such as RotDelta address their targeted bottlenecks while matching theoretical bounds? (§6.3)
- **RQ3: Serving Latency.** How effectively does two-level grouping eliminate dependency traversal waits and accelerate historical checkpoint recovery under varying network bandwidths? (§6.4)
- **RQ4: Production and Economic Benefit.** How effective is ALC when deployed on production-scale checkpoints, and what financial return does it deliver to enterprises by trading CPU compute for storage reduction? (§6.5)
- **RQ5: Generality.** How effectively does ALC generalize to non-Adam training regimes such as the emerging Muon optimizer, as well as broader numerical workloads with frequent sign flips? (§6.6)

6.1 Experimental Setup

Setup. Experiments are conducted on a server equipped with an Intel Xeon Platinum 8469C processor (3.0 GHz) and 234 GB of RAM. Our scheme is implemented using Python 3.11 and PyTorch 2.10 (CUDA 12.8). The core routines for direct/delta encoding and byte operations are implemented in C++, while checkpoint loading and compression scheme selection are handled in Python.

Models and training. We evaluate six mainstream model architectures spanning diverse design paradigms, including Qwen3.5-35B-A3B (hybrid attention Sparse Mixture-of-Experts), DeepSeek-7B (dense Transformers), Gemma4-E4B (dense Transformers with Per-Layer Embeddings), Mamba-2.8B (State Space Models), RetNet-2.7B (Retention Networks), and RWKV-7-2.9B (Attention-free RNNs). For each architecture, we perform full-parameter, mixed-precision training with the AdamW optimizer under DeepSpeed ZeRO Stage 2 on NVIDIA H20 GPUs. The complete training state, including BF16 model weights, FP32 master weights, and the first- and second-moment estimates (m_t, v_t), is periodically checkpointed at saving intervals $\Delta\text{step} \in \{100, 500, 1000\}$, with each training rank independently persisting its corresponding shard.

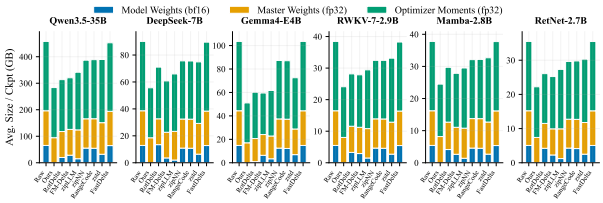


Figure 14: Compression breakdown by checkpoint component ($\Delta\text{step} = 100$).

Baselines. We compare our scheme against six lossless compression baselines, including (1) FM-Delta [37], which applies Δ-PCMap encoding followed by range coding uniformly across all parameters, (2) ZipLLM [49], which computes bitwise XOR deltas and applies Zstd compression, (3) ZipNN [17], which separates IEEE 754 byte fields and Huffman-codes the exponent stream, (4) Zstd [9], which directly compresses the raw data bytes without delta encoding, (5) range coding [40], which applies a general-purpose entropy coder to the raw data stream, and (6) FastDelta [44], a general-purpose delta compressor for backup systems that exploits rolling-hash reuse. All methods are verified by bitwise compress-and-decompress checks. We leave execution metadata uncompressed across all methods due to its negligible size (§3.1). To ensure a fair comparison with delta-based baselines, compression evaluations are conducted without checkpoint grouping (§5.4) by default, while the trade-off between storage savings and serving latency by adjusting group size is specifically evaluated in §6.4.

6.2 Overall Performance

We evaluate ALC's storage and runtime effectiveness against the baselines under representative LLM workloads. Specifically, we benchmark compression ratio, throughput, and robustness across diverse architectures, model scales, and saving intervals (Δstep). ALC consistently outperforms all baselines, demonstrating its viability for production-scale checkpointing.

Baseline comparison. Table 2 summarizes the compression ratio and throughput across the six models. ALC attains the lowest compression ratio under every configuration, beating the strongest baseline FM-Delta by 5.6–11.4 percentage points and other model or general compressors (ZipLLM, ZipNN, Range coding, Zstd) by up to 34.1 percentage points, reaching 50.2% on Gemma4-E4B at $\Delta\text{step} = 100$. Meanwhile, FastDelta achieves negligible compression as byte-level rolling hashes cannot capture floating-point updates. On throughput, ALC reaches 198.4 MB/s for compression and 232.5 MB/s for decompression, outperforming FM-Delta by 1.6× and 2.7×, respectively. Figure 14 provides a component-level breakdown at $\Delta\text{step} = 100$ (see Appendix D.1 [1] for $\Delta\text{step} = 500$). ALC eliminates derivable BF16 compute weights, removing approximately 14.3% of the raw checkpoint footprint, and further lowers the compression ratio of FP32 master weights and optimizer states by 1.95–2.61 percentage points over FM-Delta across all six models.

Table 2: Total storage and compression ratio (%) across model architectures and delta steps. Total training steps = 3000. Orig. shows total uncompressed size of all checkpoints. Compression ratio = compressed total / original total ×100. Lower is better.

Δ Step	Model	Num.	Orig. (GB)	ALC	FM-Delta [37]	ZipLLM [49]	ZipNN [17]	Range coding [40]	Zstd [9]	FastDelta [44]
100	Qwen3.5-35B-A3B	30	13732	**62.0**	70.0	74.6	84.5	85.0	85.1	98.9
	DeepSeek-7B	30	2703	**61.8**	67.4	73.2	84.0	83.8	83.1	99.4
	Gemma4-E4B	30	3106	**50.2**	57.5	59.6	84.3	84.1	70.1	100.0
	RWKV-7-2.9B	30	1154	**62.6**	72.5	76.6	84.4	84.4	86.2	99.5
	Mamba-2.8B	30	1133	**64.8**	73.7	78.1	85.0	85.2	86.5	99.9
	RetNet-2.7B	30	1065	**62.6**	71.0	77.0	83.5	83.9	85.4	99.9
500	Qwen3.5-35B-A3B	6	2746	**66.8**	76.9	80.2	86.7	87.0	87.2	99.3
	DeepSeek-7B	6	541	**65.1**	72.8	77.3	86.2	86.1	85.5	99.4
	Gemma4-E4B	6	621	**55.1**	64.6	65.7	86.5	83.9	74.2	100.0
	RWKV-7-2.9B	6	231	**65.8**	76.5	79.9	86.5	86.5	88.1	99.5
	Mamba-2.8B	6	227	**67.7**	79.1	81.9	87.1	87.2	88.4	99.3
	RetNet-2.7B	6	213	**65.8**	76.4	80.7	85.8	83.9	87.4	99.6
Throughput (MB/s)										
	Compress	—	—	198.4	121.3	397.8	355.3	217.0	523.7	72.0
	Decompress	—	—	232.5	86.8	543.8	404.8	164.0	584.9	1780.0

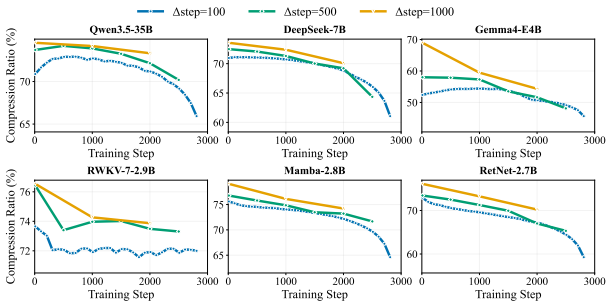


Figure 15: Compression ratio vs. training progress under different checkpoint saving intervals (Δstep ∈ {100, 500, 1000}).

Impact of training progress and saving intervals. As shown in Figure 15, the compression ratio decreases monotonically over training by 1.5% to 14% from early to late stages across all architectures, as parameters and optimizer states gradually stabilize. Moreover, once training stabilizes, increasing Δstep from 100 to 1000 raises the compression ratio by only 1% to 5%, confirming that ALC captures long-range correlations without per-stage or per-interval tuning.

6.3 Ablation and Design Analysis

6.3.1 Ablation study. Table 3 isolates the individual contributions of Phase 1 structural deduplication and Phase 2 adaptive path selection on Qwen3.5-9B. Disabling deduplication increases the overall compression ratio from 58.70% to 65.80% at Δstep = 100 and from 59.55% to 66.93% at Δstep = 500, confirming that omitting derivable BF16 weights yields an immediate reduction of over 7 percentage points. Furthermore, adaptive path selection provides significant complementary gains beyond deduplication. Enforcing any single static path degrades the compression ratio by 1.81 to 8.35 percentage points at Δstep = 100 and by 1.87 to 7.50 percentage points at Δstep = 500. Even relative to the strongest single-path baseline, RotDelta + Dedup, ALC achieves an additional 1.81 to 1.87 percentage point reduction by dynamically routing heterogeneous tensors to their lowest-cost paths. These results demonstrate that structural deduplication and adaptive multi-path encoding provide complementary gains to minimize the checkpoint footprint.

Table 3: Ablation study of ALC components on Qwen3.5-9B. We report the overall compression ratio (%).

Configuration	Dedup	Encoding Path	Δstep = 100	Δstep = 500
Direct + Dedup	✓	Direct only	67.05	67.05
FM-Delta + Dedup	✓	FM-Delta* only	60.70	61.64
RotDelta + Dedup	✓	RotDelta only	60.52	61.42
Adaptive	×	Adaptive	65.80	66.93
Adaptive + Dedup (ALC)	✓	Adaptive	**58.70**	**59.55**

6.3.2 Necessity and accuracy of adaptive selection. The necessity of adaptive compression stems from parameter heterogeneity, while its effectiveness depends on the accuracy of cost estimation. As shown in Figure 16 at Δstep = 100, evaluating all six models reveals distinct optimal encoding paths across state categories. For FP32 master weights and second-moment states v_t, Δ-PCMap consistently achieves the lowest bounds due to small updates and rare (or no) negativity (Figure 6 and Figure 8). First-moment states m_t favor Δ-RotMap and I-Align due to higher sign-flip rates and larger updates. Appendix D.2 [1] illustrates the corresponding analysis for Δstep = 500, showing that larger intervals shift preference toward direct encoding due to increased update magnitudes. Crucially, across all tensor types and models, the estimated theoretical bounds closely track the actual compressed ratios with an empirical gap of only 1% to 2%. This high estimation accuracy confirms that our cost model reliably reflects real compression performance, enabling ALC to identify the optimal encoding path for each tensor without trial compression.

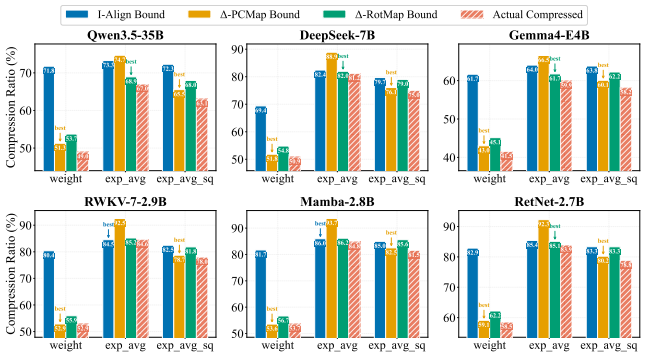


Figure 16: Theoretical bounds and actual compression ratios across state categories and models at Δstep = 100.

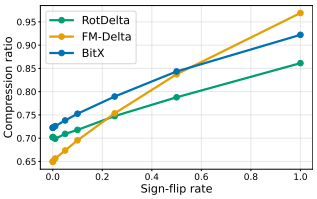


Figure 17: Compression ratio of m_t under controlled sign-flip rates.

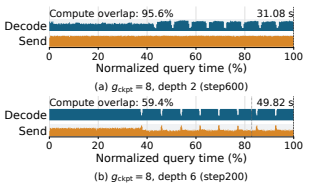


Figure 18: Pipelined decoding and transmission timeline on Qwen3.5-2B (g_{ckpt} = 8).

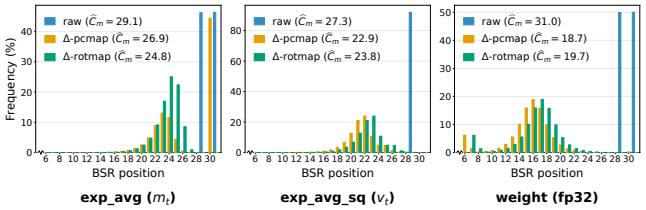


Figure 19: The distributions of BSR of the raw data, Δ-PCMap, and Δ-RotMap.

6.3.3 Benefits of RotDelta under sign flips. To validate the superiority of RotDelta, Figure 17 compares RotDelta with FM-Delta and ZipLLM under controlled sign-flip rates on first-moment states of Qwen3.5-9B checkpoint pair (Δstep = 100). RotDelta consistently achieves lower compression ratios for flip rates above 10%. For the real first-moment states m_t (at 50% flips as characterized in Figure 8), RotDelta widens its advantage to at least 6.5 percentage point reduction over baselines.

To explain this compression advantage on first-moment states, Figure 19 examines the distribution of the leading-one bit position K, computed using Bit Scan Reverse (BSR), where smaller values reflect more leading zeros and higher compressibility. By relocating the sign bit to the least significant bit, Δ-RotMap shifts the BSR distribution of m_t toward lower positions, substantially reducing the proportion of high BSR values above 28. This achieves a lower bit cost bound $\widehat{C}_m$ (from Equation (9)) of 24.8 bits per element, which the downstream leading-zero encoder (§4.2) translates directly into a superior compression ratio on m_t as shown in Figure 17.

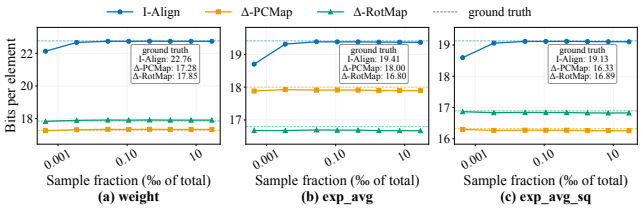


Figure 20: Convergence of sample-based compression ratio estimates across sampling rates on Qwen3.5-9B (Δstep = 100).

6.3.4 Accuracy and efficiency of subsampling. We further evaluate how sampling rate affects cost estimation accuracy and overhead. As shown in Figure 20, on a 2.35-billion-element tensor from Qwen3.5-9B, the sample-based estimate converges to the full-tensor baseline at a sampling rate of merely 0.01‰, while completing in under 1 ms.

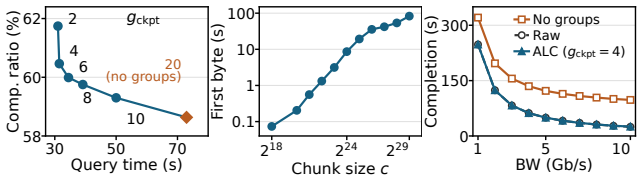
6.4 Checkpoint Serving Performance

We evaluate checkpoint serving on 20 consecutive Qwen3.5-2B checkpoints (steps 100 to 2000, Δstep = 100) using 16 CPU cores (14 decoding threads and 2 communication threads). Unless varied, we set $g_{\text{ckpt}} = 4$, chunk size $c = 2^{18}$ elements, and network bandwidth to 8 Gb/s. To enable fair comparison and isolate the decoding-transmission pipeline from underlying disk variance, archives are preloaded in memory, and query completion time spans from decompression initiation until the client receives and verifies the reconstructed FP32 and BF16 states.

Checkpoint group size. Figure 21a evaluates the trade-off between archive compression ratio and mean query completion time across $g_{\text{ckpt}} \in \{2, 4, 6, 8, 20\}$, where $g_{\text{ckpt}} = 20$ disables grouping. Increasing g_{ckpt} from 2 to 8 reduces the archive compression ratio from 61.74% to 59.75%, while mean query completion time rises from 31.03 s to 39.14 s due to longer delta chains. Specifically, at $g_{\text{ckpt}} = 4$, ALC achieves a 60.46% compression ratio with a 31.47 s completion time, incurring only 1.55% overhead over the 30.99 s raw transmission lower bound.

Chunk size. Figure 21b measures the time-to-first-byte latency when querying step 1000 from the step 1200 anchor (two delta hops) across chunk sizes $c \in \{2^{18}, 2^{20}, \ldots, 2^{29}\}$. Reducing c from 2^{29} to 2^{18} elements lowers time-to-first-byte latency from 82.87 s to 73.47 ms, as finer chunk granularity enables network transmission to start immediately after decoding a small initial slice.

Overall serving time. Figure 21c compares end-to-end serving time when querying step 1000 across network bandwidths from 1 to 10 Gb/s. Without grouping, recovery traverses ten sequential delta hops from step 2000 and incurs a 72.9 s waiting time before transmission starts. In contrast, grouped ALC ($g_{\text{ckpt}} = 4$) bounds traversal to two hops from step 1200 and pipelines decoding with transmission. Consequently, grouped ALC closely matches the uncompressed Raw baseline, taking 49.65 s versus 49.58 s at 5 Gb/s and 25.40 s versus 24.80 s at 10 Gb/s (only 2.4% overhead), achieving 1.29× to 3.85× speedups over ungrouped serving.



(a) Ckpt groups (b) Chunk size (c) Serving time

Figure 21: Checkpoint serving performance on Qwen3.5-2B.

Pipelining Analysis. Under the same setup at $g_{\text{ckpt}} = 8$, Figure 18 traces the decoding and transmission timeline across recovery depths. At two hops, 95.6% of decoding time overlaps with network transmission, yielding a 31.08 s completion time that nearly matches the 30.99 s transmission lower bound. At six hops, the overlap ratio drops to 59.4% and completion time rises to 49.82 s. This contrast validates Proposition 2, confirming that shallow recovery depths keep decoding hidden behind network transmission, whereas deeper chains shift the bottleneck to CPU computation.

6.5 Production Workload and Economic Benefit

6.5.1 Production Workload Evaluation. We further evaluate ALC on five production Qwen models ranging from 8B to 235B parameters (*e.g.*, Qwen3-MoE-235B and Qwen2.5-72B) deployed on Alibaba Group's storage infrastructure. As shown in Table 5 (Appendix D.3 [1]), ALC consistently outperforms all lossless baselines, achieving compression ratios of 56.4%–62.7% (58.1% on Qwen3-MoE-235B) and beating the strongest baseline FM-Delta by 8.2–10.1 percentage points. Across all five models, ALC reduces the total footprint from 50.2 TB to 29.4 TB, saving over 20 TB of persistent storage without per-model tuning. These results confirm that ALC remains effective on production-scale checkpoints that are substantially larger and more heterogeneous than the benchmarks.

6.5.2 Economic Benefit. To quantify when persistent storage savings outweigh the CPU costs of compression and query serving, we model the break-even retention horizon as

$$\tau^{\star} \simeq \frac{p_c/B_c}{(1-r)p_s - \lambda\kappa p_c/B_d}, \tag{12}$$

where r is the compression ratio, B_c and B_d are per-core compression and decoding throughputs, p_c and p_s are unit CPU and storage prices, λ is the query rate per checkpoint, and $\kappa = (g_{\text{ckpt}}+1)/2$ is the expected traversal depth. This also yields the maximum sustainable read rate $\lambda_{\max} = \frac{(1-r)B_d p_s}{\kappa p_c}$ (full derivation in Appendix C [1]).

Table 4 evaluates 100 PB of historical checkpoints at $\lambda = 10$ reads per checkpoint-month, with $r = 0.6$, $B_c = 200$ MB/s/core, $B_d = 232.5$ MB/s/core, $g_{\text{ckpt}} = 4$, and $p_c = \$0.15$/core-day. Across realistic storage prices [21], $\tau^{\star}$ spans only 12.6 minutes to 2.22 hours, while $\lambda_{\max}$ supports hundreds to thousands of monthly reads. Because production archives reach the exabyte scale (Figure 2) and are retained for months or years, even a 1% compression improvement saves hundreds of thousands to millions of dollars annually, confirming the substantial economic return of ALC.

6.6 The Generality of ALC

6.6.1 Performance of ALC on Muon Optimizer. We further evaluate the generality of ALC on checkpoints produced by the emerging Muon optimizer [31], which has been rapidly adopted alongside Adam in recent leading models including Qwen3.8-Flash-Next [38], Kimi K3 [24], DeepSeek-V4-Flash [10], and Moonlight [2]. We fine-tune Moonlight-16B-A3B using `MuonWithAuxAdam`, which applies Muon to hidden weight matrices and AdamW to the remaining parameters, and evaluate 10 consecutive checkpoints totaling 1.6 TB at $\Delta\text{step} = 100$. ALC achieves the lowest overall compression ratio of 48.10%, outperforming FM-Delta (51.20%), ZipLLM (56.51%), and ZipNN (73.90%). This advantage stems from adaptively choosing Direct and RotDelta encoding on Muon momentum tensors to minimize representation entropy, surpassing FM-Delta by 7.92 percentage points on momentum states. Beyond Muon momentum, ALC also consistently outperforms FM-Delta across all FP32 states by 3.75 percentage points (59.88% vs 63.63%) while surpassing ZipLLM (70.27%) and ZipNN (75.79%), driven by adaptive path selection and structural redundancy removal (§6.2).

Table 4: Illustrative economics for 100 PB of raw, compressible checkpoint history at ten reads per checkpoint-month.

Storage Price ($/GB/month)	Annual Saving ($/year)	Break-even (10 reads/ckpt/month)	$\lambda_{\max}$ (reads/ckpt/month)
0.0075	3,600,000	2.22 h	161
0.015	7,200,000	1.08 h	321
0.030	14,400,000	31.7 min	643
0.075	36,000,000	12.6 min	1,607

6.6.2 Application of RotDelta in High Sign-Flip Scenarios. RotDelta generalizes beyond LLM checkpoints to effectively compress broader numerical workloads. We evaluate an FP64 adaptation of RotDelta on two public datasets from FCBench [7], namely num-control from weather data assimilation [5] and Jane Street financial time-series.

With a 25.78% sign-flip rate on num-control, RotDelta achieves a compression ratio of 87.88%, outperforming FM-Delta and fpzip at 88.97%. On Jane Street in original row order with a 29.70% flip rate, RotDelta achieves 88.11%, outperforming FM-Delta (91.12%) and a CPU reference port of MPC (91.47%). When compressing columns independently, the sign-flip rate increases to 44.60%, where RotDelta attains 86.24%, expanding its advantage to 5.78 and 3.94 percentage points over FM-Delta and MPC, respectively. These results confirm that RotDelta provides a general-purpose delta encoding primitive for numerical data with frequent sign oscillations. Appendix D.4 [1] provides the complete comparison.

7 CONCLUSION

In this paper, we systematically characterize LLM training checkpoints and propose ALC, an adaptive lossless compression framework. ALC combines verifiable mixed-precision deduplication with subsample-guided path selection, incorporating our sign-agnostic RotDelta algorithm to suppress sign-flip inflation and a two-level grouping mechanism for low-latency historical serving. Evaluations across six LLM architectures and 235B-scale production workloads show that ALC consistently outperforms six lossless baselines while accelerating historical checkpoint serving. In future work, we plan to extend ALC with GPU-accelerated codecs (*e.g.*, [53, 59, 61]) to further accelerate the compression and decompression of large-scale checkpoints.

REFERENCES

[1] [n.d.]. ALC Full Paper. https://github.com/jerryp1/ALC/blob/main/ALC_full.pdf. Accessed: 2026-10-20.

[2] [n.d.]. Moonlight-16B-A3B. https://huggingface.co/moonshotai/Moonlight-16B-A3B

[3] Amey Agrawal, Sameer Reddy, Satwik Bhattamishra, Venkata Prabhakara Sarath Nookala, Vidushi Vashishth, Kexin Rong, and Alexey Tumanov. 2024. Inshrinkerator: Compressing Deep Learning Training Checkpoints via Dynamic Quantization. In *Proceedings of the 2024 ACM Symposium on Cloud Computing, SoCC 2024, Redmond, WA, USA, November 20-22, 2024*. ACM, 1012–1031. https://doi.org/10.1145/3698038.3698553

[4] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2397–2430.

[5] Martin Burtscher and Paruj Ratanaworabhan. 2009. FPC: A High-Speed Compressor for Double-Precision Floating-Point Data. *IEEE Trans. Computers* 58, 1 (2009), 18–31. https://doi.org/10.1109/TC.2008.131

[6] Hoyeon Chang, Jinho Park, Seonghyeon Ye, Sohee Yang, Youngkyung Seo, Du-Seong Chang, and Minjoon Seo. 2024. How do large language models acquire factual knowledge during pretraining? *Advances in neural information processing systems* 37 (2024), 60626–60668.

[7] Xinyu Chen, Jiannan Tian, Ian Beaver, Cynthia Freeman, Yan Yan, Jianguo Wang, and Dingwen Tao. 2024. FCBench: Cross-Domain Benchmarking of Lossless Compression for Floating-Point Data. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1418–1431. https://doi.org/10.14778/3648160.3648180

[8] Yu Chen, Zhenming Liu, Bin Ren, and Xin Jin. 2020. On Efficient Constructions of Checkpoints. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research)*. PMLR, 1627–1636. http://proceedings.mlr.press/v119/chen20m.html

[9] Yann Collet and Murray S. Kucherawy. 2018. Zstandard Compression and the application/zstd Media Type. *RFC* 8478, 1–54. https://doi.org/10.17487/RFC8478

[10] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 https://arxiv.org/abs/2606.19348

[11] Peter Deutsch. 1996. GZIP file format specification version 4.3. *RFC* 1952 (1996), 1–12. https://doi.org/10.17487/RFC1952

[12] Jarek Duda. 2009. Asymmetric numeral systems. *CoRR* abs/0902.0271 (2009). arXiv:0902.0271 http://arxiv.org/abs/0902.0271

[13] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, Amar Phanishayee and Vyas Sekar (Eds.). USENIX Association, 929–943. https://www.usenix.org/conference/nsdi22/presentation/eisenman

[14] Xin Gao, Sibasish Acharya, Sihui Han, Yongxiong Ren, Yanli Zhao, Liang Luo, Chucheng Wang, Pradeep Fernando, Saurabh Mishra, Siqi Yan, Yicong Du, Elzbieta Krepska, Intaik Park, Min Ni, Qunshu Zhang, and Shen Li. 2025. DECK: Experiences on Delta Checkpointing for Industrial Recommendation Systems. *Proc. VLDB Endow.* 18, 12 (2025), 4978–4990. https://doi.org/10.14778/3750601.3750621

[15] Mikaila Gossman, Avinash Maurya, Bogdan Nicolae, and Jon Calhoun. 2026. Understanding LLM Checkpoint/Restore I/O Strategies and Patterns. In *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region Workshops*. 263–273.

[16] Zhiwei Hao, Jianyuan Guo, Li Shen, Yong Luo, Han Hu, Guoxia Wang, Dianhai Yu, Yonggang Wen, and Dacheng Tao. 2025. Low-Precision Training of Large Language Models: Methods, Challenges, and Opportunities. *CoRR* abs/2505.01043. https://doi.org/10.48550/ARXIV.2505.01043 arXiv:2505.01043

[17] Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz, Or Ozeri, Ilias Ennmouri, Michal Malka, Sang (Peter) Chin, Swaminathan Sundararaman, and Danny Harnik. 2025. ZipNN: Lossless Compression for AI Models. In *18th IEEE International Conference on Cloud Computing, CLOUD 2025, Helsinki, Finland, July 7-12, 2025*, Rong N. Chang, Carl K. Chang, Jingwei Yang, Nimanthi Atukorala, Dan Chen, Sumi Helal, Sasu Tarkoma, Qiang He, Tevfik Kosar, Claudio A. Ardagna, Feras Awaysheh, Volker Hilt, and Yogesh Simmhan (Eds.). IEEE, 186–198. https://doi.org/10.1109/CLOUD67622.2025.00028

[18] Jiayu Hu, Feng Jin, Xianping Zhou, Kai Zhang, Zhen Shen, and Yongkang Luo. 2026. Harp: Improving {VPC} Network Availability via Efficient Failure Detection and Rerouting in Tencent Cloud. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 1669–1684.

[19] Zhenbo Hu, Xiangyu Zou, Wen Xia, Sian Jin, Dingwen Tao, Yang Liu, Weizhe Zhang, and Zheng Zhang. 2020. Delta-DNN: Efficiently Compressing Deep Neural Networks via Exploiting Floats Similarity. In *ICPP 2020: 49th International Conference on Parallel Processing, Edmonton, AB, Canada, August 17-20, 2020*, José Nelson Amaral, Lizy Kurian John, and Xipeng Shen (Eds.). ACM, 40:1–40:12. https://doi.org/10.1145/3404397.3404408

[20] David A. Huffman. 1952. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

[21] IDC. [n.d.]. Enterprise Storage Systems Market Insights. https://www.idc.com/promo/enterprise-storage-systems/. Accessed: 2026-10-20.

[22] William Kahan. 1996. IEEE standard 754 for binary floating-point arithmetic. *Lecture Notes on the Status of IEEE* 754, 94720-1776 (1996), 11.

[23] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. 2019. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322* (2019).

[24] Kimi Team. 2026. Kimi K3: Open Frontier Intelligence. arXiv:2607.24653 https://arxiv.org/abs/2607.24653

[25] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1412.6980

[26] Apostolos Kokolis, Michael Kuchnik, John Hoffman, Adithya Kumar, Parth Malani, Pat Caulfield, Zijian He, Wen mei Hwu, Raghunath Nambiar, and Carole-Jean Wu. 2025. Revisiting Reliability in Large-Scale Machine Learning Research Clusters. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*.

[27] Shigang Li, Jingkun Dong, Jihao Chen, Zhi Ma, and Zhongzhe Hu. 2025. Hypertron: Efficiently Scaling Large Models by Exploring High-Dimensional Parallelization Space. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1755–1768.

[28] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proc. VLDB Endow.* 13, 12 (2020), 3005–3018. https://doi.org/10.14778/3415478.3415530

[29] Wenshuo Li, Xinghao Chen, Han Shu, Yehui Tang, and Yunhe Wang. 2024. ExCP: Extreme LLM Checkpoint Compression via Weight-Momentum Joint Shrinking. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024 (Proceedings of Machine Learning Research)*, Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR / OpenReview.net, 27575–27588. https://proceedings.mlr.press/v235/li24m.html

[30] Peter Lindstrom and Martin Isenburg. 2006. Fast and Efficient Compression of Floating-Point Data. *IEEE Trans. Vis. Comput. Graph.* 12, 5 (2006), 1245–1250. https://doi.org/10.1109/TVCG.2006.143

[31] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. 2025. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982* (2025).

[32] Ilya Loshchilov, Frank Hutter, et al. 2017. Fixing weight decay regularization in adam. *arXiv preprint arXiv:1711.05101* 5, 5 (2017), 5.

[33] Stathis Maneas, Kaveh Mahdaviani, Tim Emami, and Bianca Schroeder. 2020. A Study of SSD Reliability in Large Scale Enterprise Storage Deployments. In *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, Sam H. Noh and Brent Welch (Eds.). USENIX Association, 137–149. https://www.usenix.org/conference/fast20/presentation/maneas

[34] Kiyoshi Masui, Mandana Amiri, Liam Connor, Meiling Deng, Mateus Fandino, Carolin Höfer, Mark Halpern, David Hanna, Adam D. Hincks, Gary Hinshaw, Juan Mena Parra, Laura Newburgh, J. Richard Shaw, and Keith Vanderlinde. 2015. A compression scheme for radio data in high performance computing. *Astron. Comput.* 12 (2015), 181–190. https://doi.org/10.1016/J.ASCOM.2015.07.002

[35] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2017. Mixed Precision Training. *CoRR* abs/1710.03740. arXiv:1710.03740 http://arxiv.org/abs/1710.03740

[36] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies, FAST 2021, February 23-25, 2021*, Marcos K. Aguilera and Gala Yadgar (Eds.). USENIX Association, 203–216. https://www.usenix.org/conference/fast21/presentation/mohan

[37] Wanyi Ning, Jingyu Wang, Qi Qi, Mengde Zhu, Haifeng Sun, Daixuan Cheng, Jianxin Liao, and Ce Zhang. 2024. FM-Delta: Lossless Compression for Storing Massive Fine-tuned Foundation Models. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/7b75a7339dfb256ee4b4bec028a6890b-Abstract-Conference.html

[38] Qwen Team. 2026. Qwen3.8-Flash-Next. https://github.com/QwenLM/Qwen3.8-Flash-Next

[39] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over

100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.

[40] Michael Schindler. 1998. A Fast Renormalisation for Arithmetic Coding. In *Data Compression Conference, DCC 1998, Snowbird, Utah, USA, March 30 - April 1, 1998*. IEEE Computer Society, 572. https://doi.org/10.1109/DCC.1998.672314

[41] Raunak Shah, Zhaoheng Li, and Yongjoo Park. 2025. QStore: Quantization-Aware Compressed Model Storage. *Proc. VLDB Endow.* 19, 3 (Nov. 2025), 388–398. https://doi.org/10.14778/3778092.3778100

[42] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[43] Gopalan Sivathanu, Charles P. Wright, and Erez Zadok. 2005. Ensuring data integrity in storage: techniques and applications. In *Proceedings of the 2005 ACM Workshop On Storage Security And Survivability, StorageSS 2005, Fairfax, VA, USA, November 11, 2005*, Vijay Atluri, Pierangela Samarati, William Yurcik, Larry Brumbaugh, and Yuanyuan Zhou (Eds.). ACM, 26–36. https://doi.org/10.1145/1103780.1103784

[44] Haoliang Tan, Wenhao Ou, Xiangyu Zou, Cai Deng, Yanqi Pan, Hao Huang, Zhaoquan Gu, and Wen Xia. 2026. Once Rolling Hashing is Enough: Exploiting Rolling Hash Reuse in Delta Compression. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, Antonio Barbalace, Luo Mai, Roxana Geambasu, and Peter R. Pietzuch (Eds.). ACM, 919–933. https://doi.org/10.1145/3767295.3803596

[45] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, Xin Liu, and Chuan Wu. 2025. ByteCheckpoint: A Unified Checkpointing System for Large Foundation Model Development. In *22nd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2025, Philadelphia, PA, USA, April 28-30, 2025*, Theophilus A. Benson and Radhika Niranjan Mysore (Eds.). USENIX Association, 559–578. https://www.usenix.org/conference/nsdi25/presentation/wan-borui

[46] Borui Wan, Gaohong Liu, Zuquan Song, Jun Wang, Yun Zhang, Guangming Sheng, Shuguang Wang, Houmin Wei, Chenyuan Wang, Weiqiang Lou, Xi Yang, Mofan Zhang, Kaihua Jiang, Cheng Ren, Xiaoyun Zhi, Menghan Yu, Zhe Nan, Zhuolin Zheng, Baoquan Zhong, Qinlong Wang, Huan Yu, Jinxin Chi, Wang Zhang, Yuhan Li, Zixian Du, Sida Zhao, Yongqiang Zhang, Jingzhe Tang, Zherui Liu, Chuan Wu, Yanghua Peng, Haibin Lin, Wencong Xiao, Xin Liu, and Liang Xiang. 2025. Robust LLM Training Infrastructure at ByteDance. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP 2025, Lotte Hotel World, Seoul, Republic of Korea, October 13-16, 2025*, Youjip Won, Youngjin Kwon, Ding Yuan, and Rebecca Isaacs (Eds.). ACM, 186–203. https://doi.org/10.1145/3731569.3764838

[47] Yujie Wang, Shenhan Zhu, Fangcheng Fu, Xupeng Miao, Jie Zhang, Juan Zhu, Fan Hong, Yong Li, and Bin Cui. 2025. Spindle: Efficient distributed training of multi-task large models via wavefront scheduling. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1139–1155.

[48] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. 2023. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 364–381. https://doi.org/10.1145/3600006.3613145

[49] Zirui Wang, Tingfeng Lan, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. 2026. ZipLLM: Efficient LLM Storage via Model-Aware Synergistic Data Deduplication and Compression. In *23rd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2026, Renton, WA, May 4-6, 2026*, Srikanth Kandula and Hakim Weatherspoon (Eds.). USENIX Association, 2371–2387. https://www.usenix.org/conference/nsdi26/presentation/wang-zirui

[50] Zhiqiang Wang, Wenzhe Zhu, Zaigui Zhang, Chaomei Yan, Fan Guo, Yongkun Li, and Yinlong Xu. 2025. Amber: Towards Fast and Space-Efficient Incremental Checkpointing in Large Language Model Training. In *Proceedings of the 54th International Conference on Parallel Processing, ICPP 2025, San Diego, CA, USA, September 8-11, 2025*. ACM, 678–688. https://doi.org/10.1145/3754598.3754606

[51] Ian H. Witten, Radford M. Neal, and John G. Cleary. 1987. Arithmetic Coding for Data Compression. *Commun. ACM* 30, 6 (1987), 520–540. https://doi.org/10.1145/214762.214771

[52] Mitchell Wortsman, Tim Dettmers, Luke Zettlemoyer, Ari Morcos, Ali Farhadi, and Ludwig Schmidt. 2023. Stable and low-precision training for large-scale vision-language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/20bd42d82998bc61732c00452228e814-Abstract-Conference.html

[53] Youyang Xia, Feng Zhang, Junda Pan, Yihao Liu, Jiawei Guan, Huanchen Zhang, and Xiaoyong Du. 2026. L3: A GPU-Native Co-Designed Data Format for Learned Lossless Lightweight Compression. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD (2026), 1–27.

[54] Lingrui Xiang, Xiaofen Lu, Rui Zhang, and Zheng Hu. 2024. SSDC: A Scalable Sparse Differential Checkpoint for Large-scale Deep Recommendation Models. In *IEEE International Symposium on Circuits and Systems, ISCAS 2024, Singapore, May 19-22, 2024*. IEEE, 1–5. https://doi.org/10.1109/ISCAS58744.2024.10557880

[55] Siqi Xiang, Sheng Wang, Xiaokui Xiao, Cong Yue, Zhanhao Zhao, and Beng Chin Ooi. 2025. NeurStore: Efficient In-database Deep Learning Model Management System. *Proc. ACM Manag. Data* 3, 6, Article 344 (Dec. 2025), 26 pages. https://doi.org/10.1145/3769809

[56] Huijuan Xiao, Shixi Yang, Kai Zhang, Yinan Jing, Zhenying He, and X Sean Wang. 2025. Genie: A Lightweight Serverless Infrastructure for In-Memory Key-Value Caching With Fine-Grained and Prompt Elasticity. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[57] Erci Xu, Mai Zheng, Feng Qin, Yikang Xu, and Jiesheng Wu. 2019. Lessons and Actions: What We Learned from 10K SSD-Related Storage System Failures. In *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, Dahlia Malkhi and Dan Tsafrir (Eds.). USENIX Association, 961–976. https://www.usenix.org/conference/atc19/presentation/xu

[58] Ru Zhang, Wencong Xiao, Hongyu Zhang, Yu Liu, Haoxiang Lin, and Mao Yang. 2020. An empirical study on program failures of deep learning jobs. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 1159–1170. https://doi.org/10.1145/3377811.3380362

[59] Tianyi Zhang, Mohsen Hariri, Shaochen Henry Zhong, Vipin Chaudhary, Yang Sui, Xia Hu, and Anshumali Shrivastava. 2026. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float (DFloat11). *Advances in Neural Information Processing Systems* 38 (2026), 98966–98994.

[60] Yucheng Zhang, Hong Jiang, Dan Feng, Nan Jiang, Taorong Qiu, and Wei Huang. 2023. {LoopDelta}: Embedding locality-aware opportunistic delta compression in inline deduplication for highly efficient data reduction. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 133–148.

[61] Yongqi Zhuo, Xinyu Zeng, Huanchen Zhang, and Mingyu Gao. 2026. G-CasDec: General Cascaded Decompression on GPUs. *Proceedings of the ACM on Management of Data* 4, 4 (SIGMOD (2026), 1–26.

A DETAILED FORMULATION OF PCMAP RESIDUALS AND BOUND DERIVATION

This section records the complete formulation of PCMap residuals and the step-by-step bound derivation [37] as a baseline reference.

Let $b = n_s + n_e + n_m$, with $n_s = 1$. For each floating-point value x, let $U(x)$ be its unsigned bit-pattern interpretation, $s(x) \in \{0, 1\}$ its sign bit, and $L(x) = U(x) \bmod 2^{b-1}$ its magnitude-field integer. Equivalently, $L(x) = E(x)2^{n_m} + M(x)$, where $E(x)$ is the biased exponent and $M(x)$ is the mantissa integer, representing the concatenated exponent and mantissa fields rather than the numerical magnitude $|x|$. Write $L_p = L(x_p)$ and $L_c = L(x_c)$ for corresponding elements in the preceding and current checkpoints. PCMap maps each float to an unsigned integer preserving order:

$$\mathrm{PCMap}(x) = \begin{cases} 2^{b-1} + L(x), & s = 0, \\ 2^{b-1} - 1 - L(x), & s = 1. \end{cases} \tag{13}$$

Under this mapping, negative encodings lie below 2^{b-1} and nonnegative encodings lie at or above it, giving the exact absolute residual $\Delta_{\mathrm{PCMap}} = |\mathrm{PCMap}(x_c) - \mathrm{PCMap}(x_p)|$:

$$\Delta_{\mathrm{PCMap}} = \begin{cases} |L_p - L_c|, & s_p = s_c, \\ L_p + L_c + 1, & s_p \neq s_c. \end{cases} \tag{14}$$

The effective bit-width of a nonnegative integer residual d is formally defined as:

$$\ell(d) = \begin{cases} 0, & d = 0, \\ \lfloor \log_2 d \rfloor + 1, & d > 0, \end{cases} \qquad r_{\mathrm{PCMap}} = \ell(\Delta_{\mathrm{PCMap}}). \tag{15}$$

This definition counts all significant bits, including when d is a power of two. For finite, nonzero normal inputs, let $R = \frac{\max(|x_p|,|x_c|)}{\min(|x_p|,|x_c|)}$ denote the magnitude ratio. When signs match ($s_p = s_c$), bounding the exponent difference by $q = |E_p - E_c| < \log_2 R + 1$ yields $|L_p - L_c| \le 2^{n_m}(q+1) - 1 < 2^{n_m}(\log_2 R + 2)$, which leads to $r_{\mathrm{PCMap}} \le \lceil n_m + \log_2(\log_2 R + 2) \rceil$. Under a sign flip ($s_p \neq s_c$), $\Delta_{\mathrm{PCMap}} = L_p + L_c + 1 < 2^b$, which saturates at b bits even if $|L_p - L_c|$ is small.

B PROOFS AND DERIVATIONS FOR ROTDELTA BOUNDS

B.1 Derivation of the Unified Bit-Width Bound

We derive the unified bit-width upper bound for Δ_{RotMap} across both same-sign and opposite-sign pairs.

Same-sign scenario ($s_p = s_c$). The shared LSB sign bit cancels under subtraction:

$$\Delta_{\mathrm{RotMap}} = |u_c - u_p| = 2|L_c - L_p|. \tag{16}$$

This corresponds to an exact 1-bit left shift of the same-sign PCMap delta $|L_c - L_p|$. Under finite, nonzero normal inputs, bounding the exponent difference by $q = |E_p - E_c| < \log_2 R + 1$ yields $|L_p - L_c| \le 2^{n_m}(q+1) - 1 < 2^{n_m}(\log_2 R + 2)$. Therefore,

$$\Delta_{\mathrm{RotMap}} = 2|L_c - L_p| < 2^{n_m+1}(\log_2 R + 2). \tag{17}$$

Under the effective bit-width definition $\ell(d) = \lfloor \log_2 d \rfloor + 1$ for $d > 0$ and $\ell(0) = 0$, we have $r_{\mathrm{RotMap}} = \ell(\Delta_{\mathrm{RotMap}}) \le \lceil n_m + 1 + \log_2(\log_2 R + 2) \rceil$, with $r_{\mathrm{RotMap}} = 0$ when $L_p = L_c$.

Opposite-sign scenario ($s_p \neq s_c$). For opposite signs, $(s_c - s_p) \in \{+1, -1\}$. By the triangle inequality on (5):

$$\Delta_{\mathrm{RotMap}} = |2(L_c - L_p) + (s_c - s_p)| \le 2|L_c - L_p| + 1. \tag{18}$$

Applying $|L_c - L_p| \le 2^{n_m}(\log_2 R + 2) - 1$:

$$\Delta_{\mathrm{RotMap}} \le 2\big(2^{n_m}(\log_2 R + 2) - 1\big) + 1 = 2^{n_m+1}(\log_2 R + 2) - 1. \tag{19}$$

Hence, $\ell(\Delta_{\mathrm{RotMap}}) \le \lceil n_m + 1 + \log_2(\log_2 R + 2) \rceil$. Because $\Delta_{\mathrm{RotMap}} < 2^b$, capping the residual by the maximum word width b yields the unified bound:

$$r_{\mathrm{RotMap}} \le \min\big\{b,\ \lceil n_m + 1 + \log_2(\log_2 R + 2) \rceil\big\}, \tag{20}$$

which holds unconditionally regardless of whether a sign flip occurs.

B.2 Proof of Proposition 1

Proof. Let $A = \max(L_p, L_c)$ and $B = \min(L_p, L_c)$. For opposite signs, $|s_p - s_c| = 1$. By the triangle inequality and (14),

$$\begin{aligned} \Delta_{\mathrm{RotMap}} &= \big|2(L_p - L_c) + (s_p - s_c)\big| \\ &\le 2(A - B) + 1 \\ &< A + B + 1 = \Delta_{\mathrm{PCMap}}, \end{aligned} \tag{21}$$

where the strict inequality follows from $A < 3B$. This argument includes equal nonzero magnitude fields ($A = B > 0$) and does not require choosing an ordering of the inputs. □

C DETAILED ECONOMIC AND COST-BENEFIT DERIVATION

This section formalizes the storage-to-compute economic model introduced in §6.5.2, deriving the break-even retention horizon and sustainable query thresholds.

Cost components. Let V denote the raw byte volume of historical checkpoints eligible for compression, excluding the latest uncompressed recovery checkpoint $C^{(T)}$. Let r represent their aggregate compressed-to-raw size ratio including Direct anchors, B_c the effective per-core compression throughput, p_c the unit price of CPU core-time, p_s the storage price per byte per unit time, and τ the retention horizon. The one-time compression cost incurred during archival is

$$C_{\mathrm{comp}} = \frac{V}{B_c} \cdot p_c. \tag{22}$$

Over retention duration τ, the cumulative storage expenditure saved by ALC is

$$C_{\mathrm{stor}}(\tau) = (1 - r) \cdot V \cdot p_s \cdot \tau. \tag{23}$$

Query serving and decompression costs. Serving a historical checkpoint requires decoding its group's Direct anchor and sequentially traversing d delta hops, where $0 \le d \le g_{\mathrm{ckpt}} - 1$ (§5.4). At an effective per-core decoding throughput B_d, each hop decodes one raw-checkpoint-equivalent volume. If λ denotes the mean query frequency per checkpoint per unit time, the cumulative expected decoding cost over retention horizon τ is

$$C_{\mathrm{dec}}(\tau) \simeq \lambda \tau \kappa \frac{V}{B_d} p_c, \tag{24}$$

where $\kappa = \mathbb{E}[1 + d]$ denotes the expected number of decoding operations per query. Under uniformly distributed historical queries within full ckpt groups, $\kappa = (g_{\mathrm{ckpt}} + 1)/2$.

Break-even horizon and sustainable query rate. Compression achieves positive financial return whenever cumulative storage savings exceed total compute expenditure

$$C_{\text{stor}}(\tau) \;>\; C_{\text{comp}} + C_{\text{dec}}(\tau). \tag{25}$$

Substituting (22), (23), and (24) yields

$$(1-r)Vp_s\tau \;>\; \frac{V}{B_c}p_c + \lambda\tau\,\kappa\frac{V}{B_d}p_c. \tag{26}$$

Dividing both sides by V and isolating τ gives

$$\tau\left((1-r)p_s - \lambda\kappa\frac{p_c}{B_d}\right) \;>\; \frac{p_c}{B_c}. \tag{27}$$

Assuming a positive net saving rate per unit time, the break-even retention horizon is

$$\tau^\star \;\simeq\; \frac{p_c/B_c}{(1-r)p_s - \lambda\kappa p_c/B_d}. \tag{28}$$

To ensure that $\tau^\star > 0$, the net storage savings per time unit must exceed recurrent decoding expenses, which establishes the maximum sustainable query rate

$$\lambda_{\max} \;=\; \frac{(1-r)B_d p_s}{\kappa p_c}. \tag{29}$$

When $\lambda \geq \lambda_{\max}$, decompression consumes more compute budget per unit time than storage saves, preventing long-term amortization. When $\lambda < \lambda_{\max}$, the economic return-on-investment ratio

$$R(\tau) \;=\; \frac{C_{\text{stor}}(\tau)}{C_{\text{comp}} + C_{\text{dec}}(\tau)} \tag{30}$$

scales proportionally with retention horizon τ, yielding immense cost savings for cold archival workloads.

D EXTENDED EXPERIMENTS

D.1 Compression breakdown with Δstep = 500

Figure 22 provides a detailed breakdown across different checkpoint components with Δstep = 500.

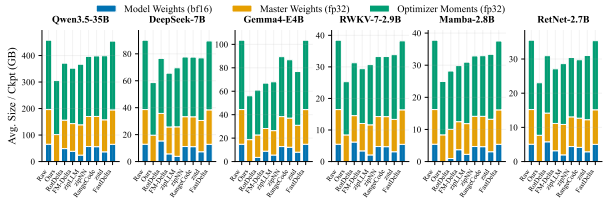


Figure 22: Compression breakdown by checkpoint component (Δstep = 500).

D.2 Necessity of Adaptive Selection at Δstep = 500

Figure 23 illustrates the necessity of the compression section when using a large checkpoint interval of Δstep = 500.

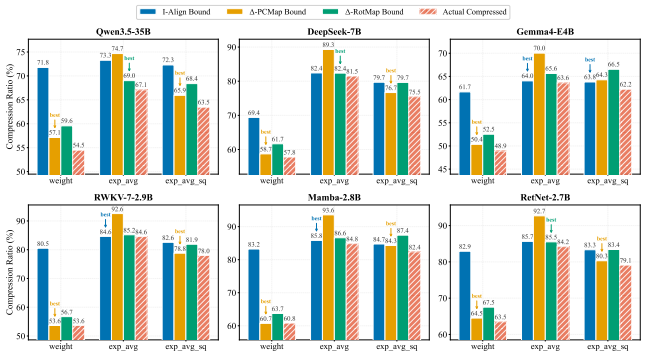


Figure 23: Theoretical bounds and actual compression ratios across state categories and models at Δstep = 500.

D.3 Comparison on Production Checkpoints

Table 5 reports the detailed storage volumes and compression ratios across the five production Qwen models evaluated in §6.5.1.

D.4 RotDelta on Public Numerical Data

The complete num-control dataset contains 19,938,093 FP64 values. Using the 136-column shape reported by FCBench, we interpret the Jane Street file in row-major order and extract four 8,192-row windows starting at zero-based rows 0, 416,130, 832,260, and 1,248,390. These windows total 35.65 MB, approximately 1.97% of the full dataset. We retain the original byte order or compress each column independently, giving every method identical input bytes within each layout. Both RotDelta and FM-Delta compute sequential deltas against the preceding element along the time dimension. Sign-flip rates count sign-bit changes between consecutive values in each encoded sequence, excluding sequence boundaries. Table 6 reports compressed bytes divided by input bytes, including stored reference values and metadata, and aggregates the four Jane Street windows by size.

These supplementary measurements use single-threaded CPU codecs on an Apple M4 Pro. For this benchmark, we adapt RotDelta and FM-Delta* to FP64 sequences using the same local tANS backend with a 256-state table. FM-Delta uses the authors' original C++ PCMap and adaptive range coder through a sequential-reference wrapper. The fpzip, ndzip, and blocked Bitshuffle+Zstd codec sources come from FCBench and are compiled locally. Bitshuffle+Zstd uses 1,024-element blocks and Zstd level 3. The global variant applies Zstd level 3 to the entire bitshuffled sequence.

The MPC row reports our experimental CPU reimplementation based on the published CUDA code. We have not executed the official GPU implementation or verified compatibility with its compressed stream. BUFF was not executed, so no compression or losslessness comparison with BUFF is established. These measurements therefore cover the tested implementations and do not constitute a full FCBench reproduction. Each configuration is encoded three times, and its saved compressed file is decoded three times and verified against the original bytes.

Table 5: Total storage and compression ratio (%) for production checkpoints. Orig. shows total uncompressed size of all checkpoints. Compression ratio = compressed total / original total $\times 100$.

Δ Step	Model	Num.	Orig. (GB)	ALC	FM-Delta [37]	ZipLLM [49]	ZipNN [17]	Range coding [40]	Zstd [9]	FastDelta [44]
2	Qwen2.5-72B	12	11,376	**60.9**	69.1	76.8	81.9	80.8	85.2	98.6
5	Qwen3-MoE-235B	5	15,959	**58.1**	67.1	73.0	82.0	81.2	79.2	98.0
10	Qwen3-MoE-30B	41	16,186	**56.4**	66.5	70.8	80.7	80.2	78.5	99.3
10	Qwen3.5-27B	18	6,436	**62.7**	71.4	73.4	81.4	81.8	81.4	98.5
22	Qwen3-VL-8B	2	216	**59.1**	67.5	71.9	80.2	82.0	84.5	98.7

Table 6: Actual compression ratios (%) on public FP64 data. Lower is better. Jane Street results cover four sampled windows. BS denotes Bitshuffle, and MPC denotes our experimental CPU reimplementation.

Method	num-control Full dataset	Jane Street Original order	Jane Street Per column
Input size (MB)	159.50	35.65	35.65
Sign-flip rate (%)	25.78	29.70	44.60
RotDelta	**87.88**	**88.11**	**86.24**
FM-Delta	88.97	91.12	92.02
FM-Delta*	90.62	91.68	92.21
fpzip	88.97	92.97	92.03
ndzip	90.19	91.99	90.30
MPC (CPU)	90.25	91.47	90.18
BS+Zstd (blocked)	89.23	92.62	91.30
BS+Zstd (global)	88.65	91.39	90.87